

中山大學
SUN YAT-SEN UNIVERSITY

并行硬件和并行软件 (Ch 2.3)

王桢

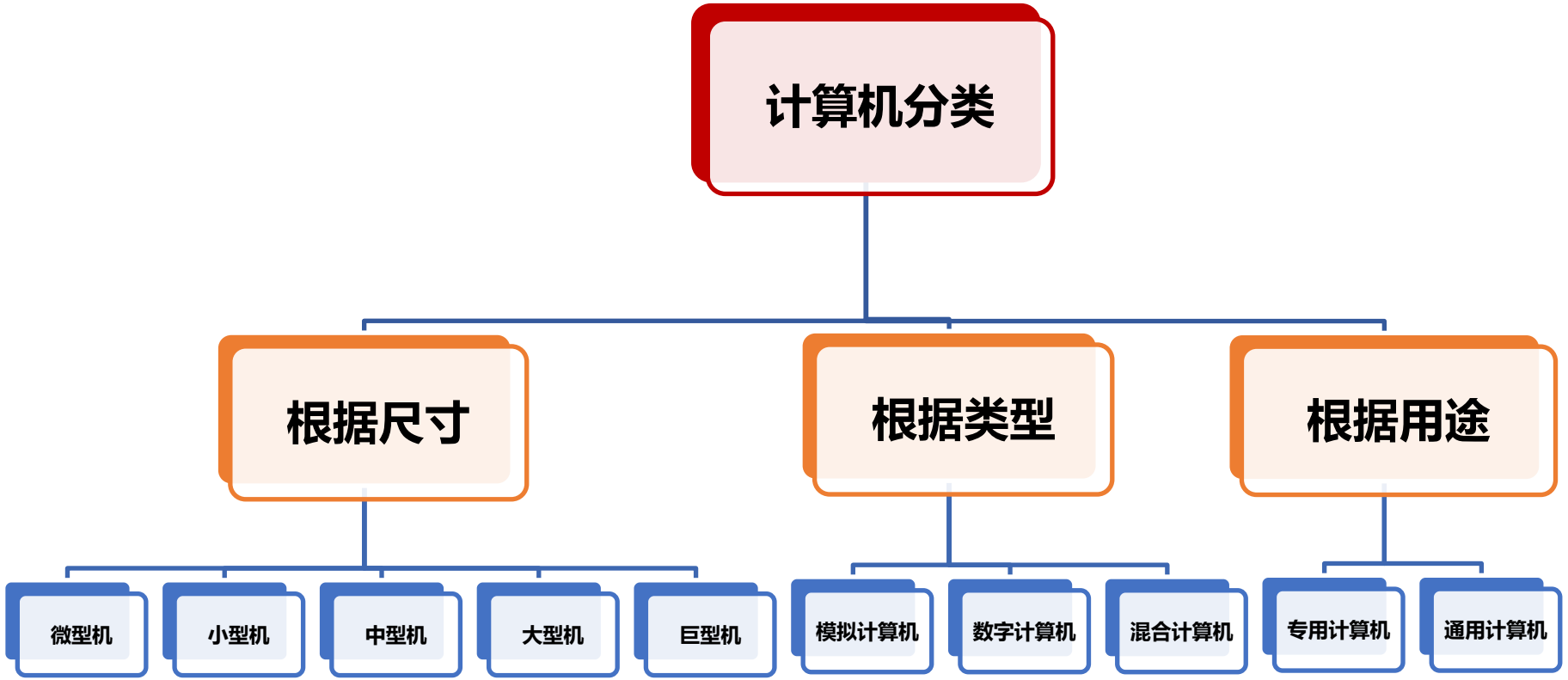
wangzh665@mail.sysu.edu.cn

目录

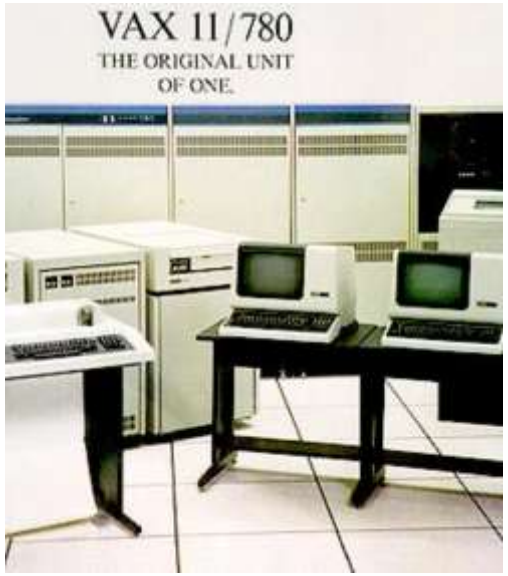
中山大學
SUN YAT-SEN UNIVERSITY

- **Classification of parallel systems**
  - **Flynn's taxonomy: classified by the number of instruction streams and the number of data streams**
  - **SIMD**
  - **MIMD**
  - **Further classified by how the cores access memory: Shared-memory vs. distributed-memory**
- Interconnection networks
- Cache coherence

传统计算机分类方法


计算机分类
根据尺寸
根据类型
根据用途
微型机
小型机
中型机
大型机
巨型机
模拟计算机
数字计算机
混合计算机
专用计算机
通用计算机

传统计算机分类方法



VAX 11(小型机)



IBM System/3(中型机)



IBM Z9(大型机)

传统分类方法的缺陷

- **存在的缺陷：**
  - 由于计算机硬件技术的发展，划分标准需随时间变化
  - 这样分类不能反映机器的系统结构特征
- **弗林分类法的提出：**
  - 为了解决原有分类方法的缺点，1972年Michael J. Flynn提出了一种基于数据流和指令流的并行性关系的分类方法

弗林介绍



- **弗林(Michael J. Flynn)：**
  - 计算机体系结构领域的开创者，并以提出Flynn分类法而闻名世界。
  - Flynn教授因对计算机和数字系统体系结构的技术贡献而于1992年获得ACM / IEEE Eckert-Mauchley奖。
  - 是五本书和三百多篇技术论文的作者，并且同时是IEEE和ACM Fellow。

弗林分类法的相关概念

- **指令流(Instruction)：**
  - 指机器执行的指令序列，即将一系列数据送入数据处理单元进行修改的命令。
- **数据流(Data)：**
  - 指由指令流调用的数据序列，包括输入数据和中间结果，但不包括输出数据。
- **多倍性(Multiple)：**
  - 指在系统性能瓶颈部件上处于同一执行阶段的指令或数据的最大可能个数。

弗林分类法

- **面向计算机体系结构的一种分类方法**
  - 基于指令和数据来划分

	Single instruction	Multiple Instruction
Single data	**SISD**	**MISD**
Multiple data	**SIMD**	**MIMD**

其中”I”和”D”分别代表指令流和数据流。每个类别又可以根据多倍性分为两类，一类用”S”表示单数据流，另一类用”M”表示多数据流。

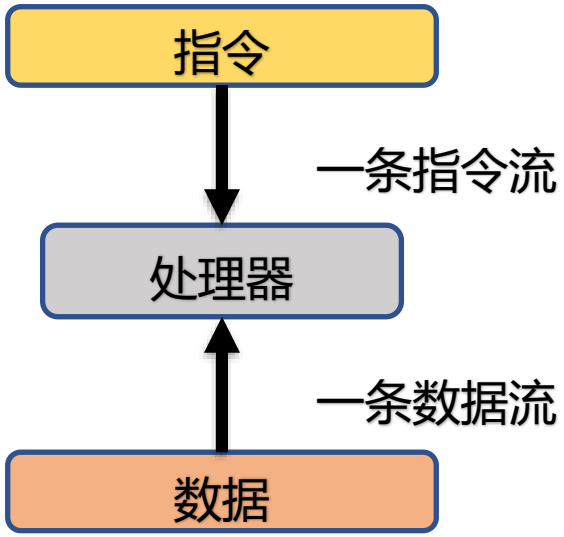
弗林分类法

名称	指令流	数据流	举例
SISD	1个	1个	传统冯诺依曼计算机
SIMD	1个	多个	向量计算机，阵列计算机
MISD	多个	1个	很少用
MIMD	多个	多个	多处理机，多计算机系统

- 另外，随着的图形处理器(GPU)近年来的不断发展，一种基于SIMD的新的架构SIMT出现了，它的全称是：SIMT-单指令流多线程流（Single Instruction stream, Multiple Threads stream），稍后介绍

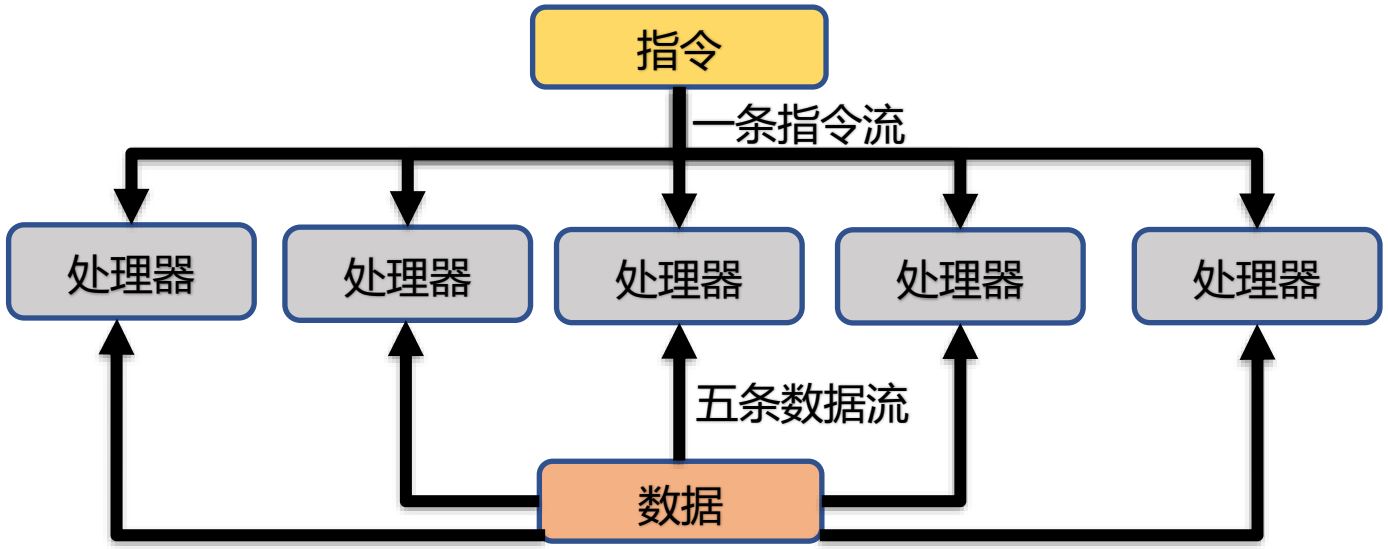
SISD体系结构

- **SISD计算机是一种传统的串行计算机**
  - 它从硬件上就不支持任何并行化计算，所有的指令都是串行执行
  - CPU在一个时钟周期内只能执行一条数据流。因此这种机器被称为单指令单数据流机器
  - 早期计算机多为SISD计算机，如早期的IBM PC机，早期的巨型机等

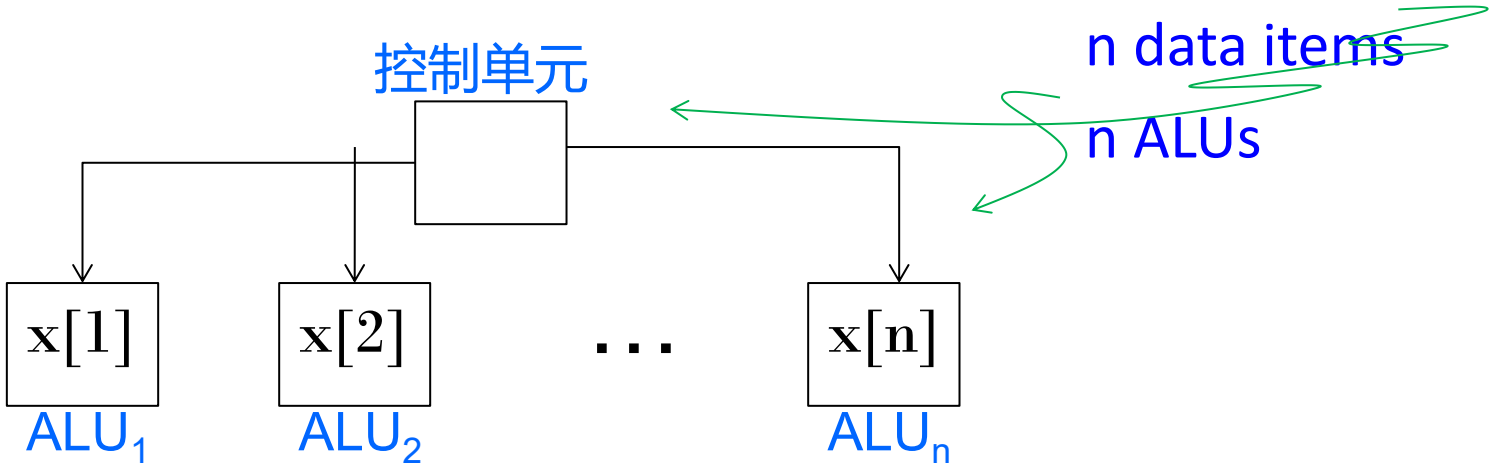


SIMD体系结构

- **SIMD计算机可以实现数据级并行，对多个不同的数据流并行执行相同的数据处理操作。**
  - 该类型计算机主要适用于解决使用向量和矩阵等复杂科学计算和大规模工程计算的问题。
  - 这类机器大多应用于数字信号处理、图像处理等领域，SIMD根据类型又可以分为阵列计算机和向量计算机



SIMD示例




```
for (i = 0; i < n; i++)
    x[i] += y[i];
```

SIMD

- **问题：**
  - 如果没有和数据项一样多的ALU，如何处理？
- **解决方法：**
  - 把任务进行切分，分批处理

Round3	ALU_1	ALU_2	ALU_3	ALU_4
1	X[0]	X[1]	X[2]	X[3]
2	X[4]	X[5]	X[6]	X[7]
3	X[8]	X[9]	X[10]	X[11]
4	X[12]	X[13]	X[14]	*Idle*

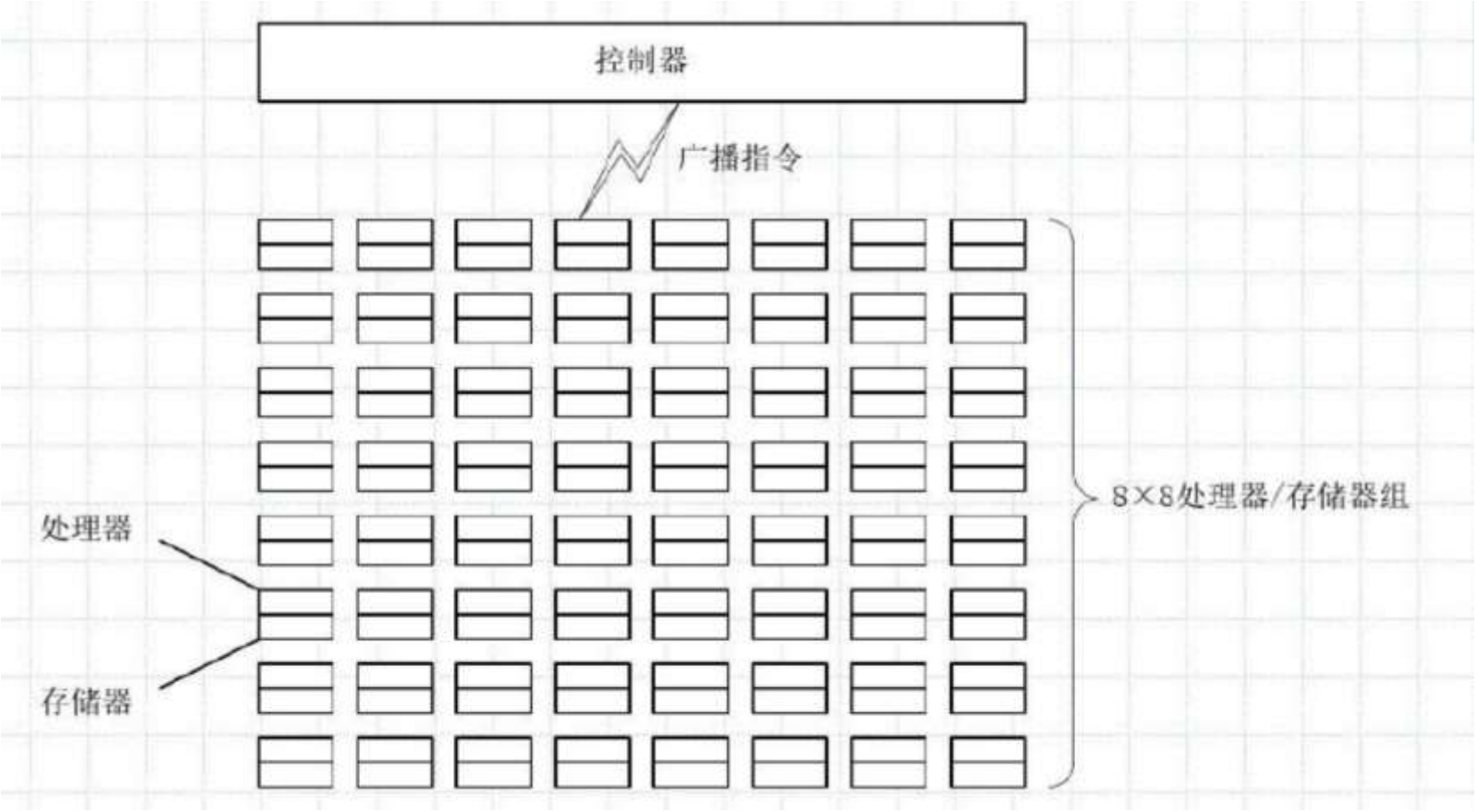
SIMD体系结构 – 阵列计算机

- **阵列计算机的基本思想是用一个单一的控制单元提供信号驱动多个处理单元同时运行**
  - 每个处理器单元都由CPU或者功能增强版的计算单元和本地内存组成。
  - 所有的处理器单元都由相同的控制单元发出的指令流控制。
  - 每个处理单元可以选择执行或者不执行控制器发出的指令流，处理单元之间通过互连网络连接。
- **目前阵列计算机的发展没有取得较大的成功，发展前景并不明朗**



美国ILLIAC-IV阵列计算机，于1960年代建造完成，1981年停用。

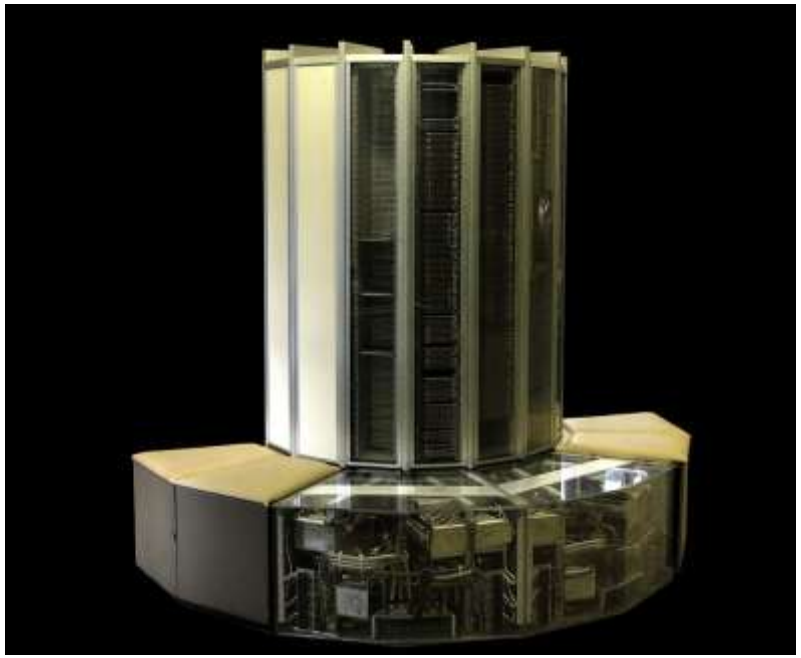
SIMD体系结构 – 阵列计算机



美国ILLIAC-IV阵列计算机基本结构

SIMD体系结构 – 向量计算机

- **向量计算机是指专门对向量进行处理的计算机**
  - 主要是以流水线结构为主
  - 以向量作为基本操作单元
  - 操作数和结果都以向量的形式存在。



第一台向量计算机Cray-1，由Cray Research公司1975年推出，此后该型号计算机在科学计算领域占据了数十年的统治地位。

SIMD体系结构 – 向量计算机



我国自主研发的第一台大型向量计算机”757”计算机于1983年交付使用，该研究成果获得了1985年国家科技进步一等奖。

- “上世纪60年代初期，科学院计算所承担了军用、民用、通用、专用、各式各样的计算机任务。任务是五花八门，但是特点都是相同的：**没有合同，没有经费。**我们从没有考虑有没有经费、报酬、日后算不算成果这些事情，一心想着认真完成上级交给的任务……”



---高庆狮院士

向量处理器 vs. 传统CPU

- **向量处理器**
  - 能够对数组或者数据向量进行操作
- **传统CPU**
  - 对单独的数据元素或者标量进行操作

向量处理器的主要部件

- **向量寄存器**
  - 能存储由多个操作数组成的向量
  - 长度：4-128个64位元素不等
- **向量化和流水化的功能部件**
  - 对向量中的每个元素执行同样的操作
- **向量指令**
  - 对向量进行操作的指令，而非标量

向量处理器的主要部件

- **交叉/交错存储器 （interleaved memory）**
  - 内存系统由多个内存体(Banks)组成
  - 每个内存体能够独立同时访问
  - Interleaved memory mainly improves **throughput**, not necessarily the latency of one access.
    - After accessing one bank, there will be a delay before it can be reaccessed, but a different bank can be accessed much sooner.
  - If too many requested elements map to the same bank, accesses collide.

向量处理器的主要部件

- **步长式/跨步式存储器访问（strided memory access）**
  - 能够访问向量中固定间隔（步长）的元素
  - 如：第1,5,9个元素
- **硬件散射/聚集（hardware scatter/gather）**
  - 指对无固定间隔的数据进行读（gather）和写（scatter）

向量处理器的优点

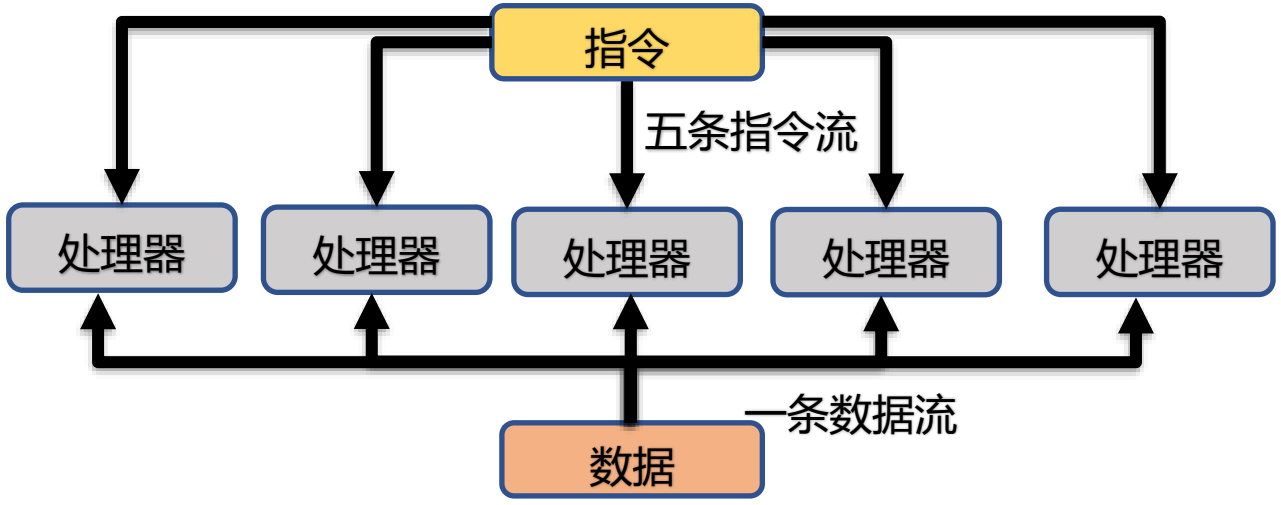
- **优点：**
  - 速度快
  - 容易使用
  - 向量编译器擅长识别向量化的代码
    - 能识别不能向量化的循环，并提供循环不能向量化的原因
    - 用户能对是否重写代码来支持向量化做出明智决定
  - 很高的内存带宽
  - 充分利用高速缓存中的每个元素

向量处理器的缺点

- **缺点：**
  - 不能处理不规则的数据结构和其他并行结构
  - 可扩展性受到限制
    - 可扩展性是指能够处理更大问题的能力
    - 新一代系统通过增加向量处理器的数目，而非增加向量长度来进行扩展
  - 对长向量的支持需要定制，非常昂贵

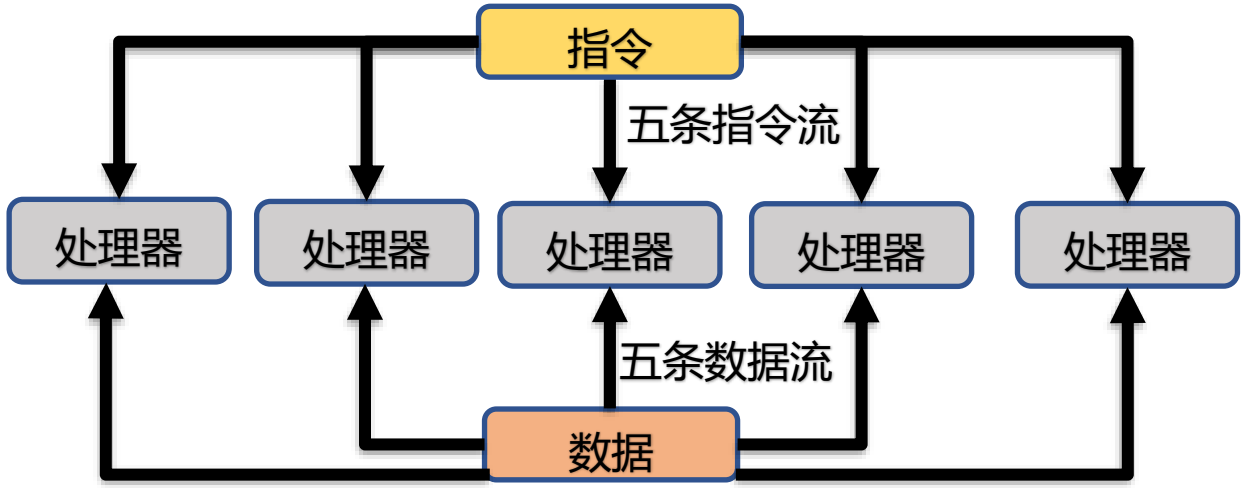
MISD体系结构

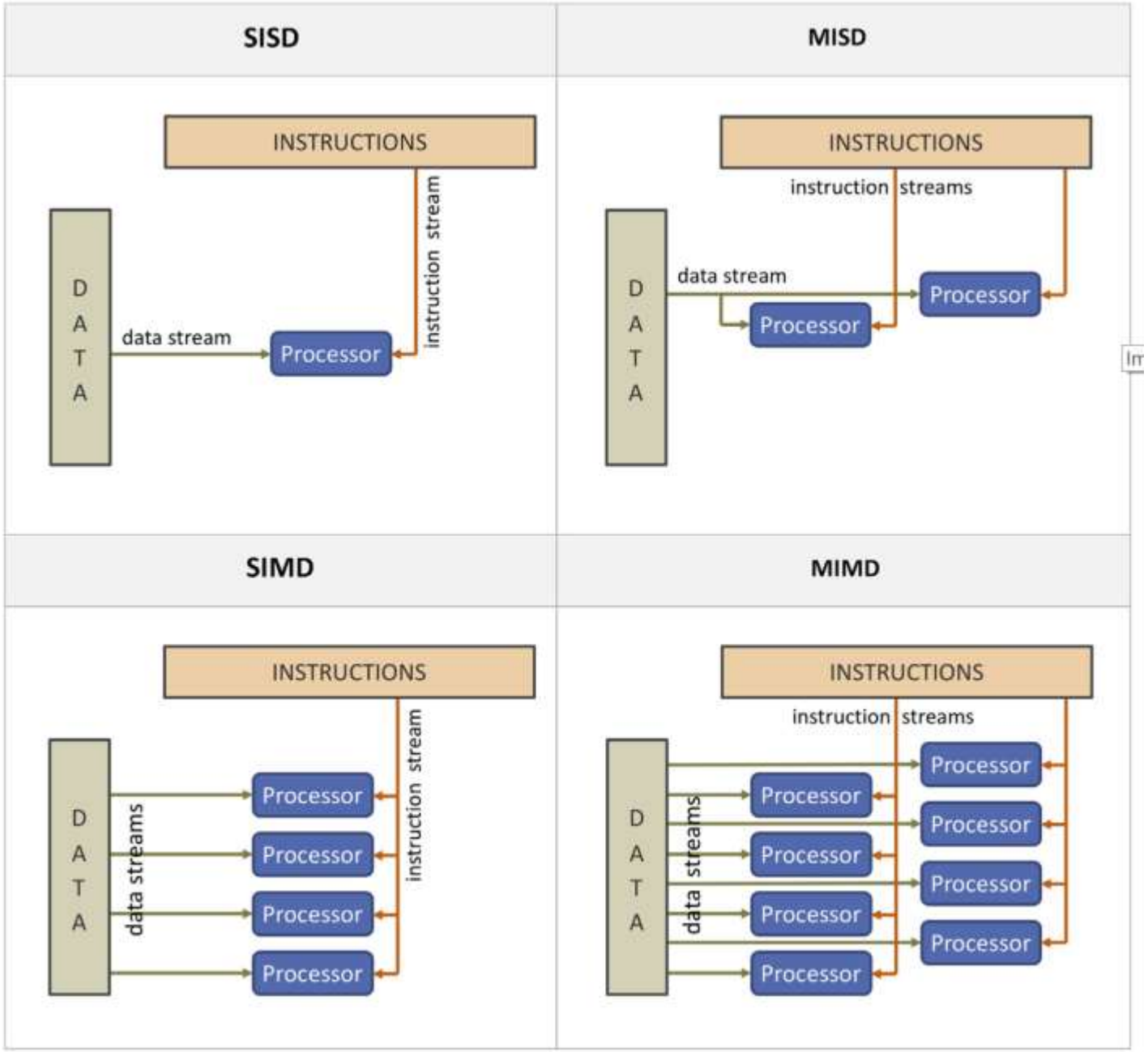
- **在MISD中，每个处理器都按照不同指令流的要求来对同一个数据流以进行不同的处理**
  - 该类型提出存在一定争议，有人认为这种类型的计算机至今都未出现。
  - 也有其他人认为有一些类似的例子，例如共享内存的多处理器系统和计算机中的流水线结构。



MIMD体系结构

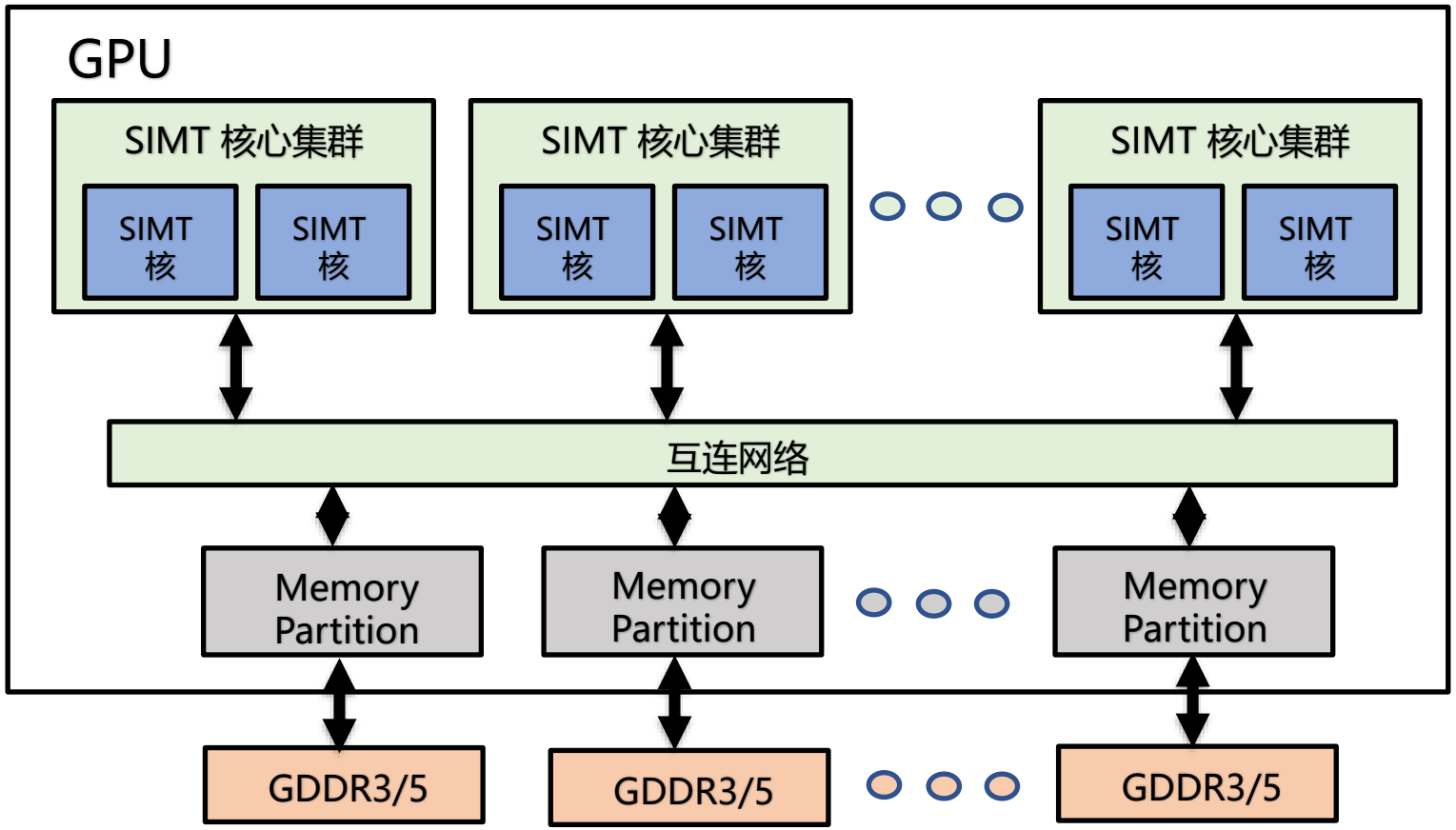
- **MIMD中的每个处理器都有自己的指令流，也可以和其他处理器共享指令流，对自己的数据进行处理**
  - 这是**最广泛应用的并行体系结构的形式**，现代流行的并行处理结构都可以划分为这一类。
  - 超算、计算机集群、分布式系统、多处理器计算机和多核计算机都划分为这种类型。




SISD
INSTRUCTIONS
instruction stream
D A T A
data stream
Processor
MISD
INSTRUCTIONS
instruction streams
D A T A
data stream
Processor
Processor
SIMD
INSTRUCTIONS
instruction stream
D A T A
data streams
Processor
Processor
Processor
Processor
MIMD
INSTRUCTIONS
instruction streams
D A T A
data streams
Processor
Processor
Processor
Processor
Processor
Processor
Processor
Processor

扩展：SIMT体系结构

- SIMT是一种并行计算中使用的模型，主要是将SIMD与多线程结合在一起，广泛应用于GPU上的计算单元中



SIMT vs. SIMD

- **SIMT比SIMD更灵活**
  - SIMT允许一条指令的多数据分开寻址
  - SIMD是必须连续在一起的片段
- **SIMD中的向量中元素相互之间可以自由通信**
  - SIMD的向量存在于相同的地址空间（例如，都在CPU的同一寄存器中）
- **SIMT中的每个线程的寄存器都是私有的**
  - SIMT线程之间只能通过共享内存和同步机制进行通信。

图形处理单元（GPU）

- **实时图形应用编程接口**
  - 使用点、线、三角形来表示物体的表面
  - 使用图形处理流水线将物体表面的内部表示转换为一个像素的数组
  - 该像素数组可以在屏幕上显示
  - 流水线的许多阶段是可编程的
    - 通过着色函数（shader）来说明

图形处理单元（GPU）

- **GPU使用SIMD并行来优化性能**
  - 对邻近元素使用相同的着色函数
  - 等同于使用相同的控制流
- **GPU严重依赖硬件多线程**
  - 避免内存访问的延迟
- **GPU不是纯粹的SIMD系统**
  - 在一个给定的核上ALU使用了SIMD并行
  - GPU有很多个核，每个核都能独立执行指令流

Flynn分类法的局限性

- **局限性：**
  - 分类的对象主要是控制驱动方式下的串行处理和并行处理计算机，对于**非控制驱动方式的计算机不适合**。
  - 把两个不同等级的功能并列对待导致**MISD计算机不存在**。
  - **分类太粗**，对流水线处理机的划分不明确，如标量流水线为SISD，向量流水线为SIMD
- **其他分类方法：**
  - 冯氏分类法：美籍华人教授冯泽云1972年提出按最大并行度来定量描述各种计算机系统
  - 1978年D.J.Kuck提出按控制流和执行流分类。
  - Wolfagan Handler在1977年提出根据并行度和流水线分类

MIMD细化分类

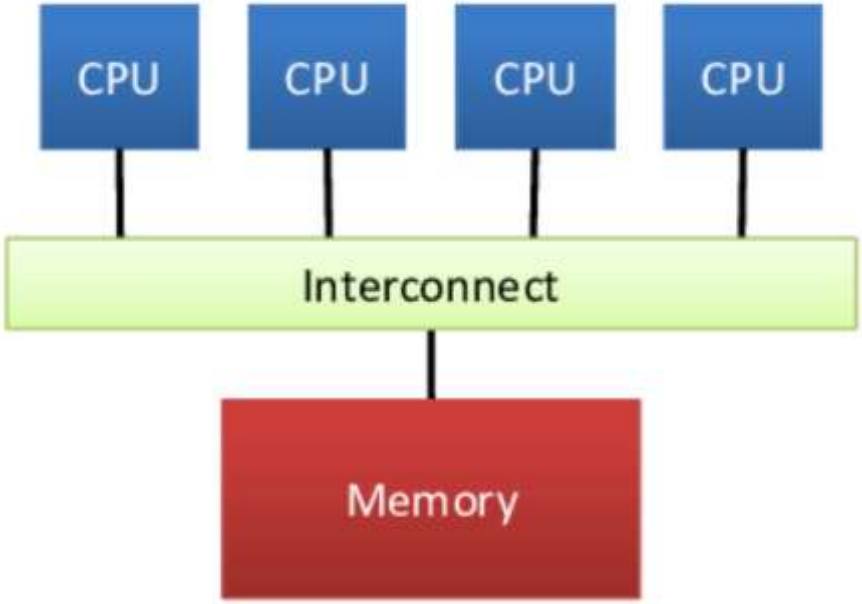
根据不同的CPU是如何组织和共享内存的，MIMD机器分为如下的两类：

- **共享式内存（Shared Memory）**
- **分布式内存（Distributed Memory）**

- 其中有些资料将**基于分布式内存的MIMD计算机**称为基于**消息驱动（Message-Passing）**的计算机

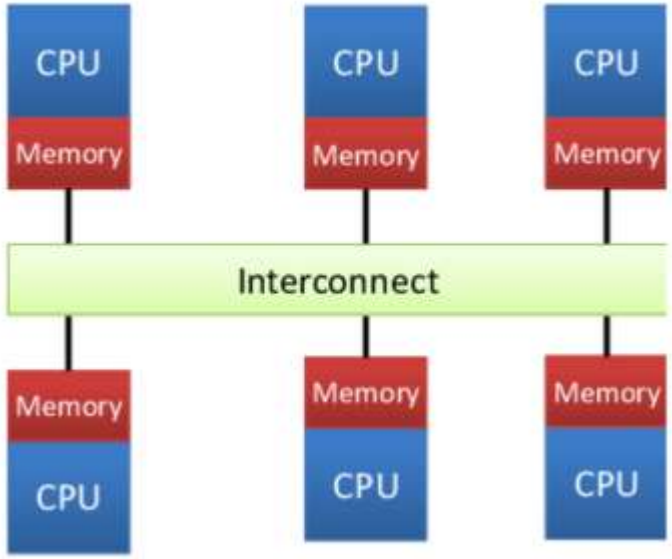
MIMD体系结构－共享式内存

- **共享式内存系统就是处理器之间共享内存，通过共享内存进行通信**
  - 所有的处理器都通过软件或者硬件的方式连接到一个“**全局可用**”的存储器。

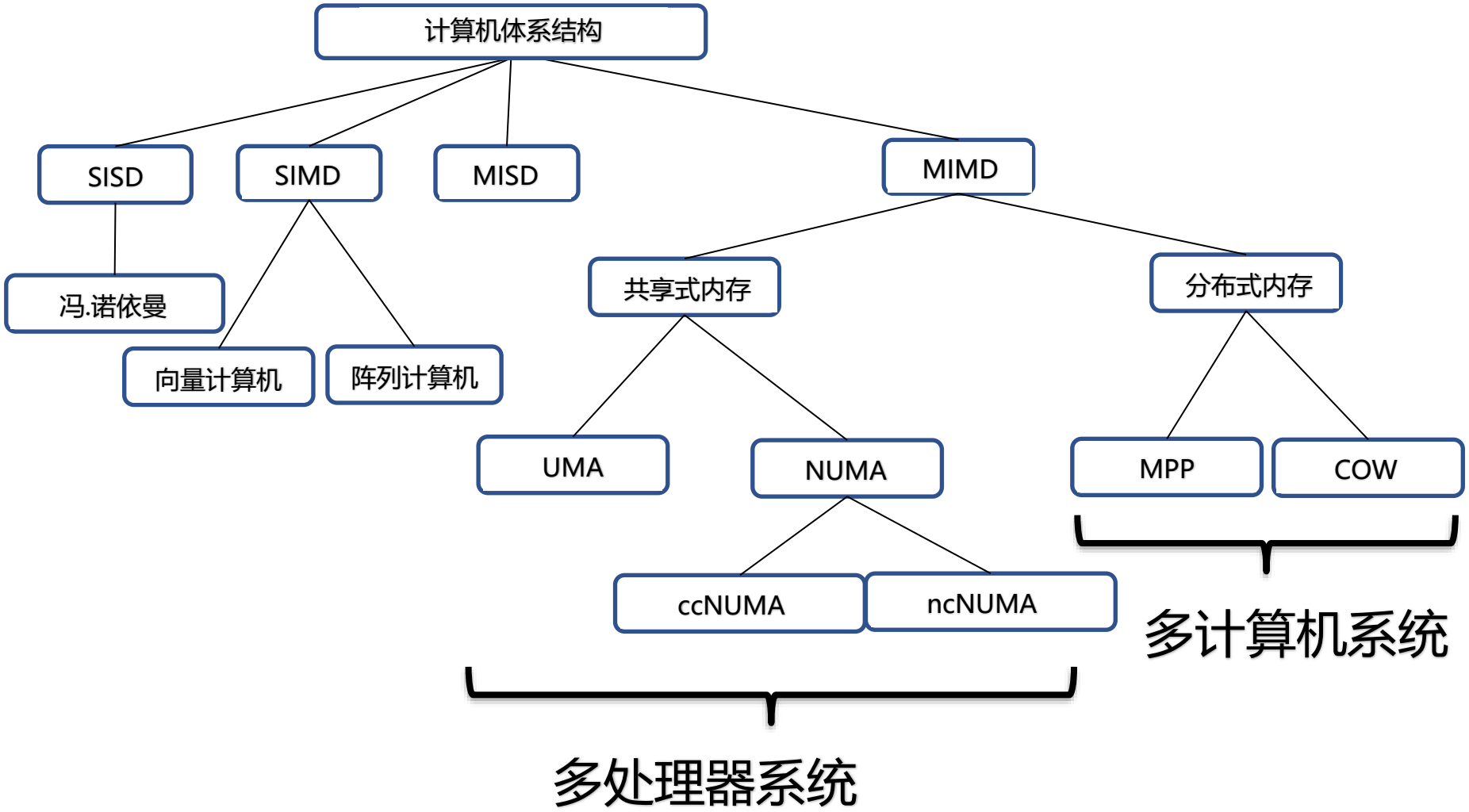


MIMD体系结构 – 分布式内存

- **分布式内存系统（消息驱动）是处理器之间不共享内存，通过消息驱动来通信**
  - 对于要共享的数据，它必须作为消息从一个处理器传递到另一个处理器
  - 如果两个处理器之间没有运行软件协议加以辅助，那么一个处理器就无法访问另一个存储器的数据。



弗林分类法的扩展


计算机体系结构
SISD
SIMD
MISD
MIMD
冯.诺依曼
向量计算机
阵列计算机
共享式内存
分布式内存
UMA
NUMA
MPP
COW
ccNUMA
ncNUMA
多计算机系统
多处理器系统

（二）共享式内存系统

共享式内存系统

共享式内存系统可以分为如下两类：

- **集中共享内存系统(CSM)**：
  - 又叫**对称多处理器系统(SMP)、一致存储访问系统(UMA)。**
- **分布式共享内存系统(DSM)**：
  - 又叫**非一致存储器访问系统(NUMA)。**

在这两种体系结构中，线程之间的通信都是通过**共享内存地址**来完成的，所谓“**共享内存**”指的就是将所有的存储器抽象成一个整体存储地址。

共享式内存系统

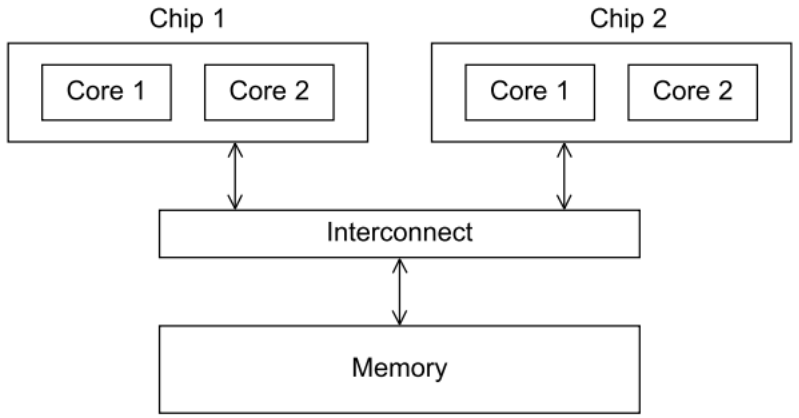


FIGURE 2.5

A UMA multicore system.

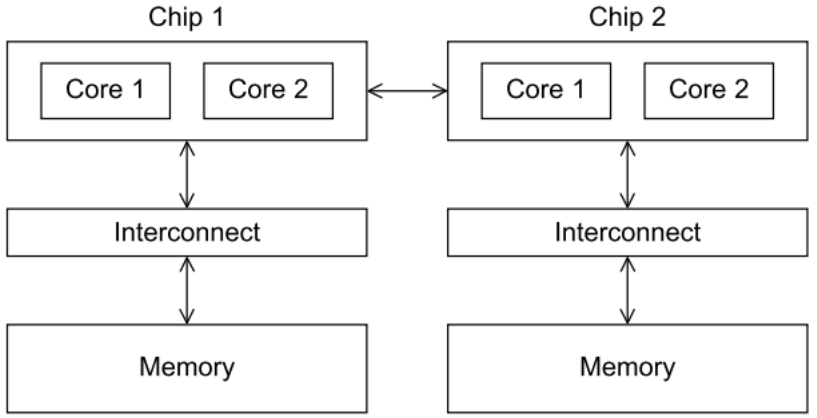


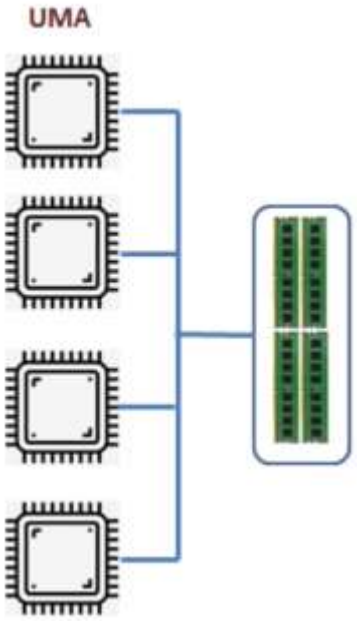
FIGURE 2.6

A NUMA multicore system.

1）集中共享式内存

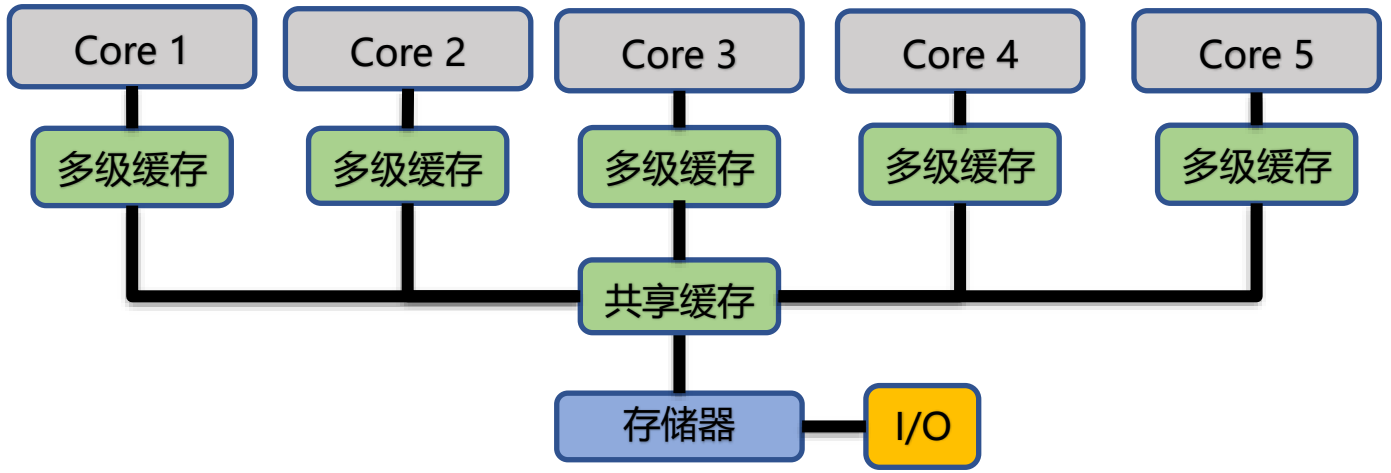
- **集中共享式内存(Centralized Shared Memory)，**
  - 又叫**对称多处理器**(Symmetric Multiprocessors)，
  - **一致存储访问系统**(Uniform Memory Access)

- 此类多处理器中处理器数目较少，所以处理器之间能够共享一个集中式的存储器
- 所有的存储器能够平等的访问它，这就是对称一词的由来。
- 又因为每个处理器都能平等地访问存储器，所以它们访问存储器的延迟都是相同的，因此又被叫做一致存储访问系统

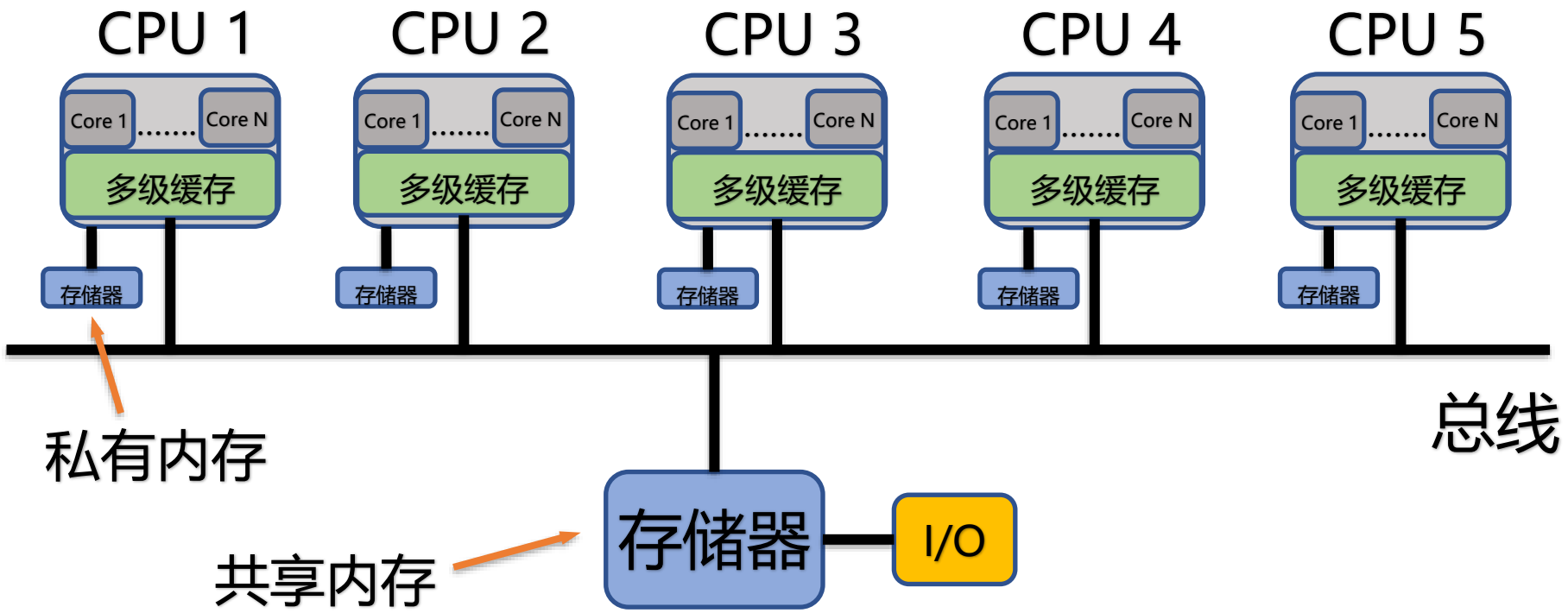


集中共享式内存系统的特点

- 有一个存储器被所有处理器均匀共享。
- 所有处理器访问共享的存储器的延迟相同。
- 每个处理器可以拥有私有内存或高速缓存。



基于多核芯片的集中共享内存系统的基本结构

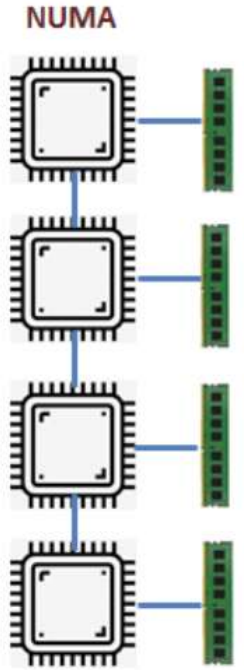


- 上图展示了**基于多个芯片的集中共享内存系统（带缓存和私有内存）**
- 与多核芯片不同，这里每个芯片可以认为是一个处理器，将所有的处理器通过总线或者互连网络连接到同一个存储器，而不是在一个芯片的内部，并且每个处理器都有自己的私有内存和共享内存。

2）分布式共享内存系统

- **分布式共享内存系统 (Distributed Shared Memory)，**
  - 又叫非一致存储访问系统(Non-Uniform Memory Access)

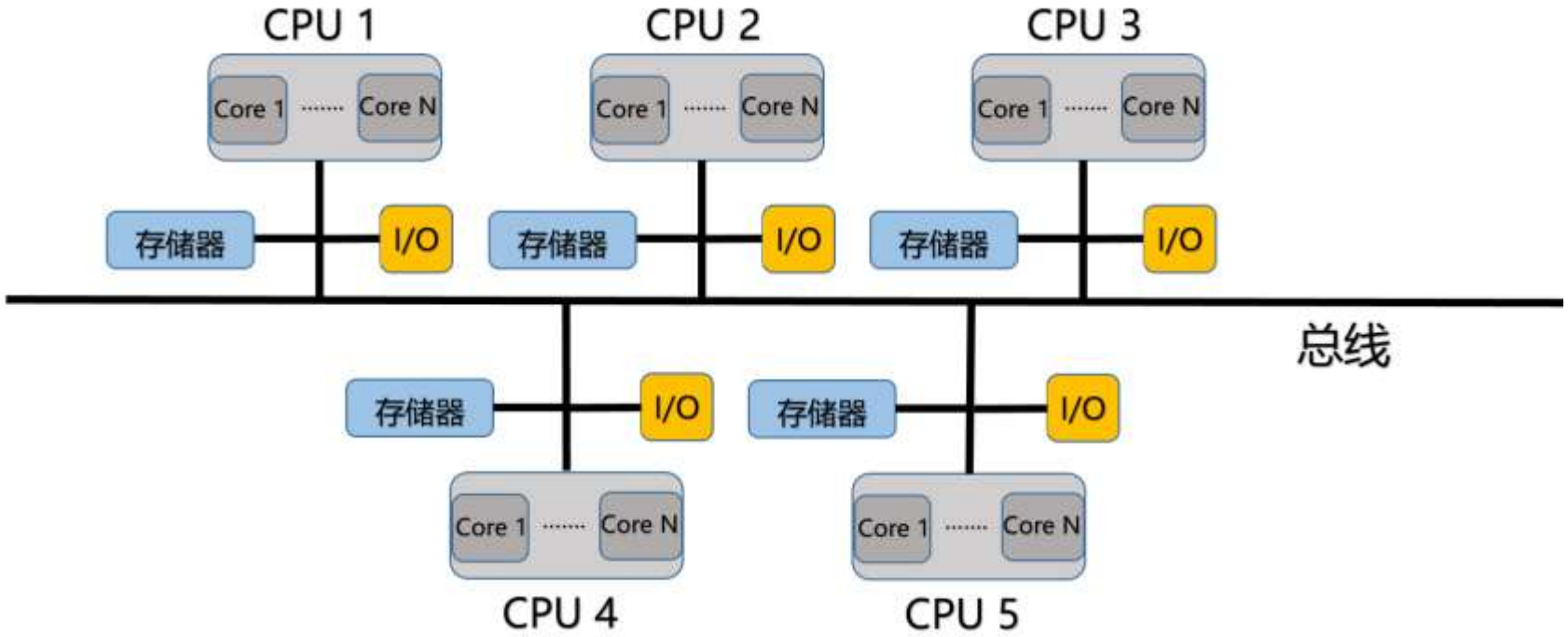
- 每个处理器都拥有自己的存储器，也可以访问其他节点的存储器
- 这类系统之所以叫做非一致存储访问系统，是因为每个结点访问本地内存和访问其它结点的远程内存的延迟是不同的。



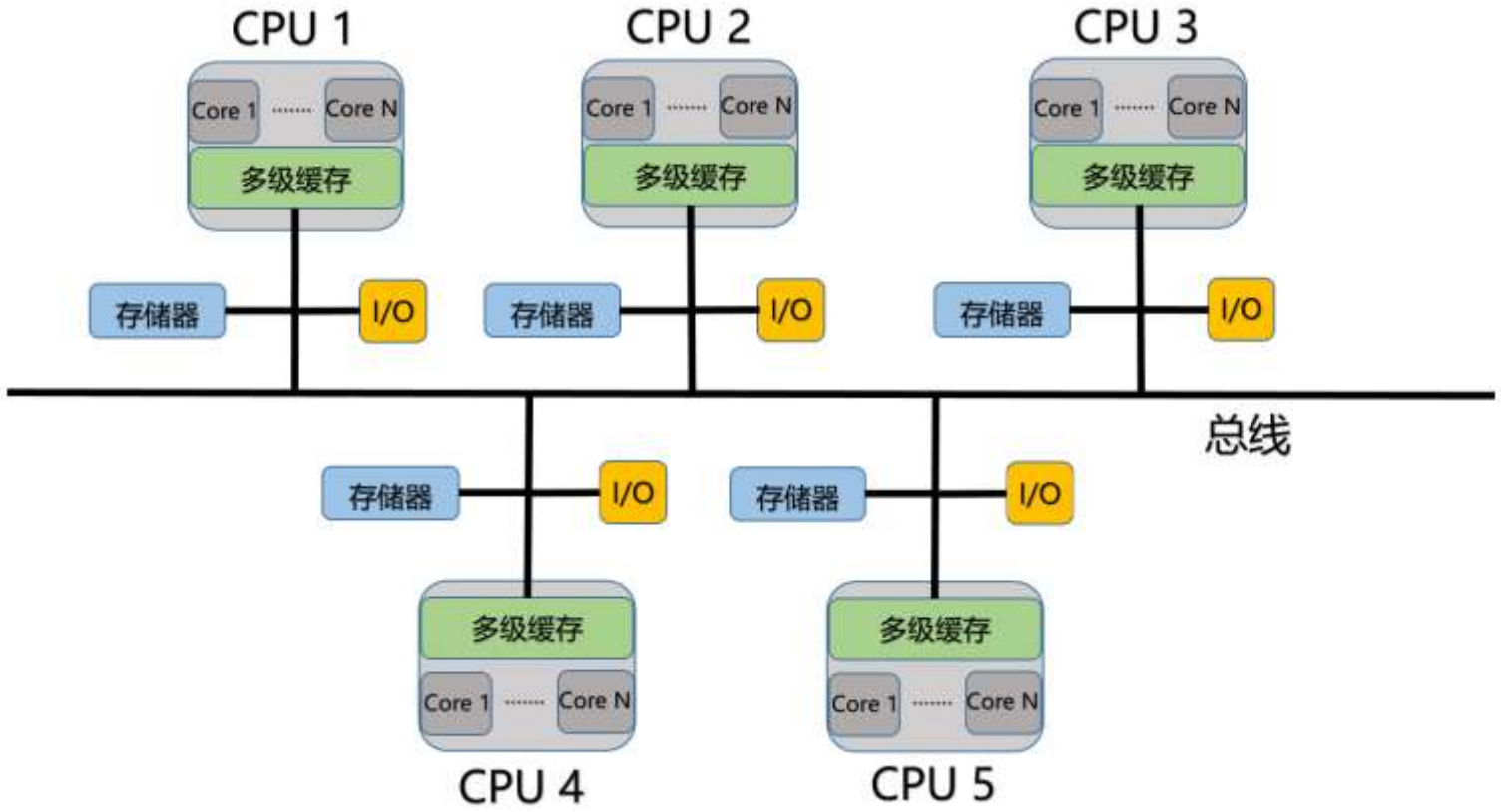
分布式共享内存系统的特点

- 所有的处理器都能访问一个单一的地址空间。
- 使用LOAD和STORE指令访问远程内存。
- 访问远程内存比访问本地内存延迟要高。
- 每个处理器可以使用高速缓存

目前几乎所有的多核心多处理器系统都使用了分布式存储器



- 上图展示了**分布式共享内存系统（不带缓存），又叫NC-NUMA(Non Cache)。**



- 上图展示了**分布式共享内存系统（带缓存），又叫CC-NUMA(Cache Coherence)**
  - 这两种类型的共享存储器在物理上是分布在不同的处理器的本地存储器之上
  - 但是他们都通过总线或者互连网络相连接，可以把所有这些本地存储**抽象成一个全局的存储器**，并能够被任何处理器访问。

（三）分布式内存系统

分布式内存系统

分布式内存系统可以分为如下两类：

① 大规模并行处理器系统(MPP)

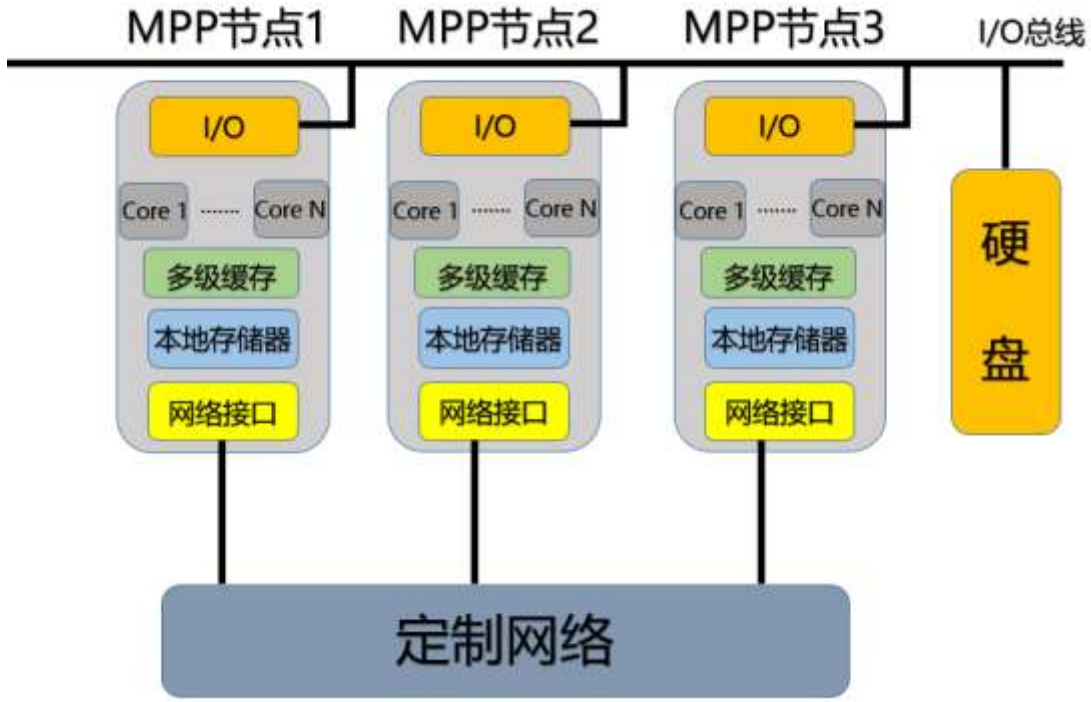
② 工作站机群系统(COW)

- 在这种计算机体系结构中，每台计算机使用**消息机制（如以太网）**连接起来
- 每台计算机都有自己的处理器，每个处理器都有**自己的私有内存**，私有内存只提供自己的处理器进行访问
- **其他的计算机不能直接访问**，每个计算机都有自己独立的物理地址空间

1）大规模并行处理系统

- **MPP：大规模并行处理器（Massively Parallel Processors）系统**
  - MPP系统是由成百上千台计算机组成的大规模并行计算机系统
  - 主要应用于科学技术、工程模拟等以计算为主的科研工作中。
  - 该系统一般开发困难，价格高，市场有限，是国家和公司综合实力的象征。

大规模并行处理系统基本架构

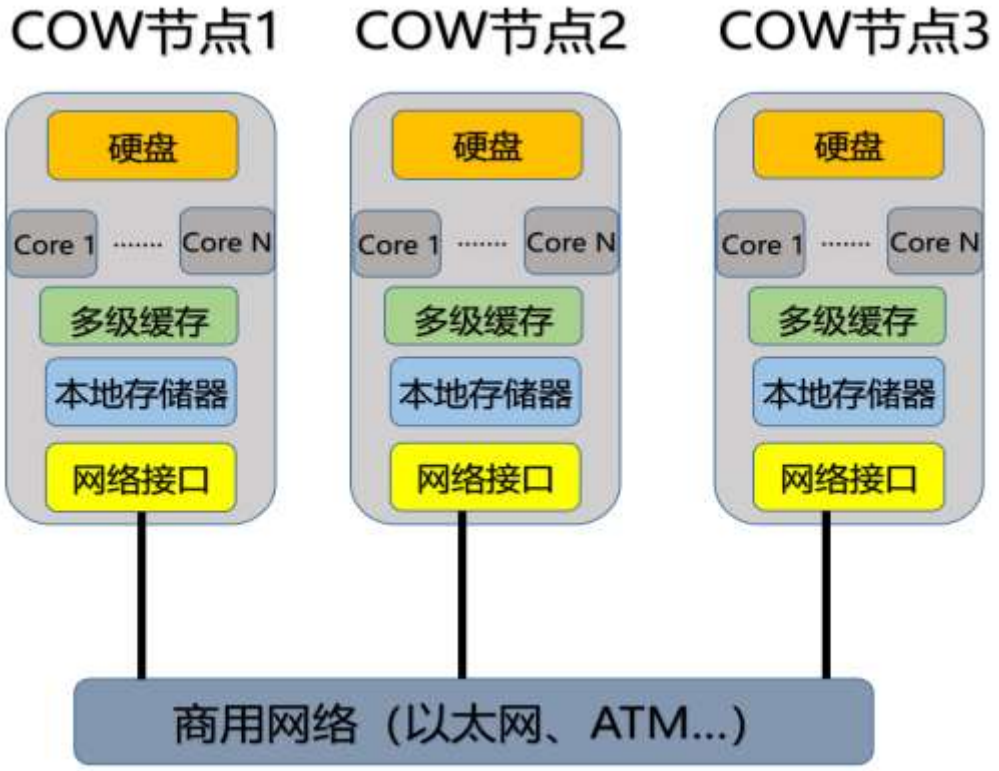


- MPP中一般每个节点可以认为是一个**没有硬盘的计算机**
- MPP节点一般只驻留操作系统内核，由一条I/O总线连接到同一个硬盘上面。
- MPP使用的网络一般情况不是我们使用的高速以太网，它一般使用制造商专有的定制高速通信网络。

2）工作站机群系统

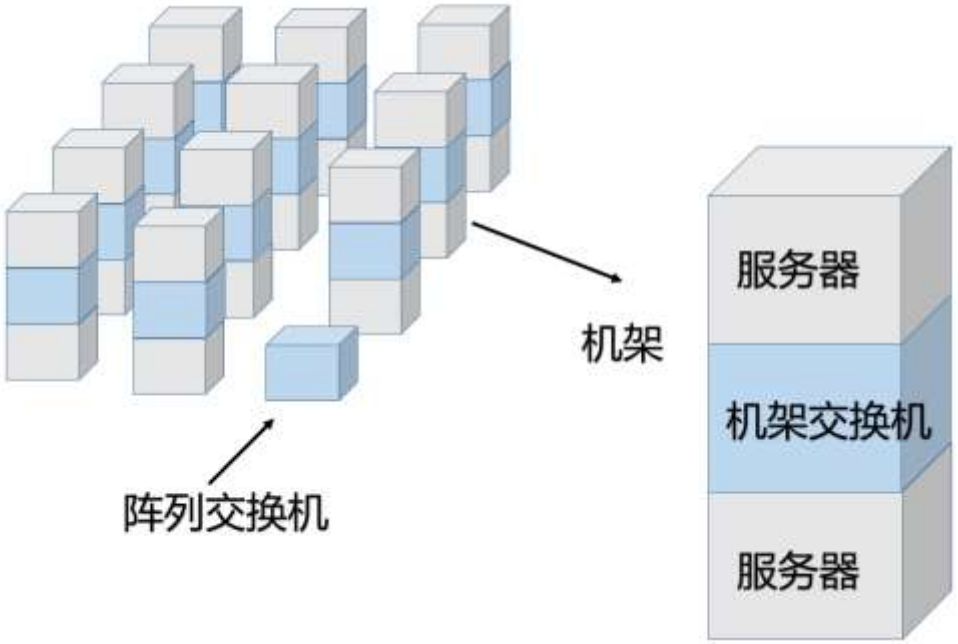
- **COW：工作站机群系统 (Cluster Of Workstations)**
  - 又叫仓库级计算机(Workstations Cluster)
  - COW系统是由大量的家用计算机或者工作站通过商用网络连接在一起而构成的多计算机系统。
    - 很多大公司的数据中心就是一个典型的例子。
  - 随着云计算的蓬勃发展，COW正在变得越来越重要
    - 相比过去为科学家和工程师们提供高性能计算的角色，现在的COW已经成为为全世界提供信息技术的基础。

工作站机群系统的基本架构



- COW中每个节点都可以认为是一台独立的计算机，它们有自己的硬盘、CPU、存储器等，在商用网络的协作下组成一个工作站机群系统。

COW的交换机层次结构



- **COW通常由多个服务器阵列排列而成。**
  - 其中机架是容纳服务器、交换机的外壳框架，一个机架上往往有多个服务器，服务器之间通过机架交换机进行通信。
- **服务器阵列是由多个机架排列而成，阵列内部机架间通过阵列交换机通信。**



腾讯云计算数据中心机房



Microsoft的水下数据中心

- COW的成本极高，它包含了机房、配电与制冷基础设施，服务器和联网设备，一般情况下一个COW能够容纳上万台服务器。



神威太湖之光超级计算机

- 根据2018年6月全球超级计算机排行榜Top500名单中，有**437台**超算采用的是COW架构，另外**63台**采用的是MPP架构。
- 前十名中，有6名使用的是MPP架构，中国的神威太湖之光使用的是MPP架构。

COW vs. HPC

- 部分HPC采用的是COW的架构，但是一般不采用商业互联网和商业芯片，HPC一般使用定制的芯片和通信网络。
- HPC强调线程级并行或数据级并行，而COW则强调请求级并行，即可能有多个网络请求同时访问一台机器。
- HPC常常满负载持续数周完成大规模运行作业，而COW是面向并发请求的，通常不会满负载。

小结

- **并行计算机的分类**
  - SISD，SIMD，MISD和MIMD
  - 其中共享内存系统又分为：集中式共享内存系统(CSM/SMP/UMA)和分布式共享内存系统(DSM/NUMA)。
  - 而分布式内存系统分为：大规模并行处理系统(MPP)和工作站机群系统(COW/WSC)

目录



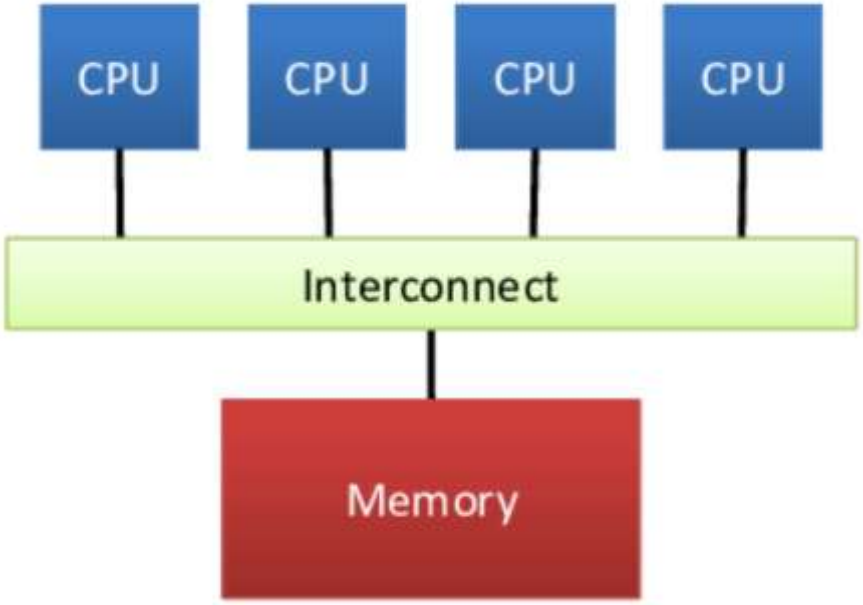
中山大學
SUN YAT-SEN UNIVERSITY

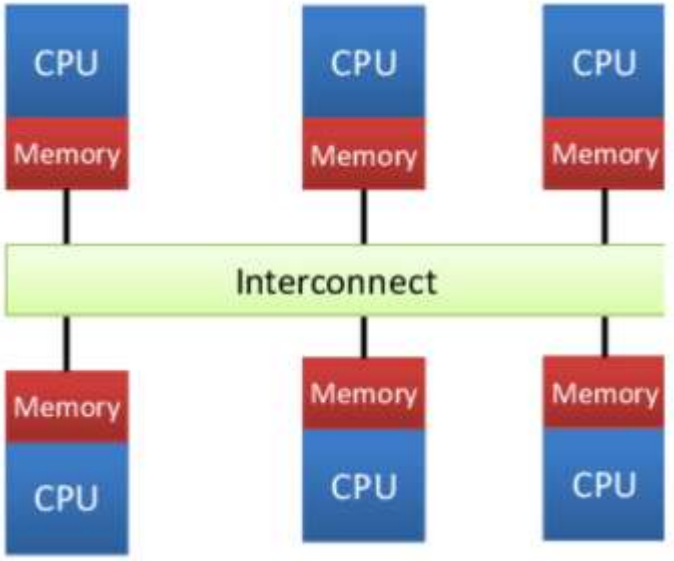
- Classification of parallel systems
- **Interconnection networks** （互连网络）
  - 分析共享内存和分布式内存互连网络
- Cache coherence

Why is it important?

中山大學
SUN YAT-SEN UNIVERSITY

- It plays a decisive role in the performance of both distributed- and shared-memory systems



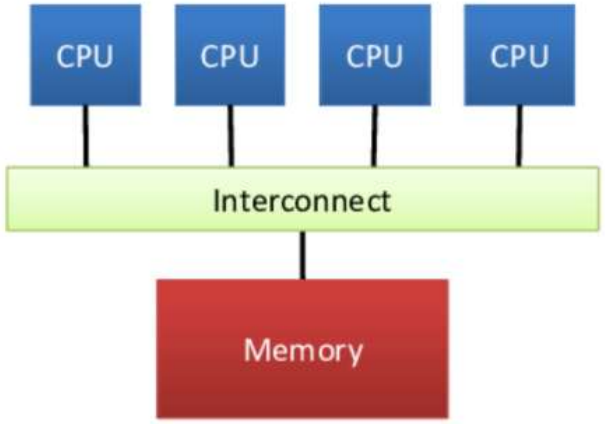


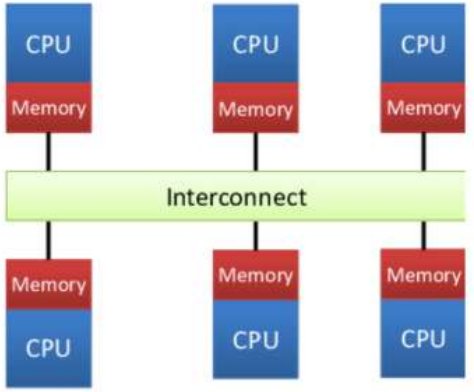
What is interconnection networks?



中山大學
SUN YAT-SEN UNIVERSITY

- **An Interconnection Network (ICN) is a programmable system that transports data between terminals**
  - To hold our parallel machines together at the core of parallel computer architecture
  - Share basic concept with LAN/WAN
  - but very different trade-offs due to very different time scale/requirements





What is interconnection networks?



中山大學
SUN YAT-SEN UNIVERSITY

- **Interconnection networks can be grouped into four domains**
  - Depending on number and proximity of devices to be
    - **Wide-Area Networks** [广域网]
    - **Local-Area Networks** [局域网]
    - **System-Area Networks** [系统区域网络]
    - **On-Chip Networks** [片上网络]

Wide-Area Networks



中山大學
SUN YAT-SEN UNIVERSITY

- Interconnect systems distributed across the globe
  - Internetworking （网间互通/网络互连） support is required
  - Millions of devices interconnected
- Maximum interconnect distance
  - many thousands of kilometers

Local-Area Networks



中山大學
SUN YAT-SEN UNIVERSITY

- Interconnect autonomous computer systems
  - Machine room or throughout a building or campus
  - Hundreds of devices interconnected
- Maximum interconnect distance
  - Few meters to tens of kilometers
  - Example (most popular):
    - Ethernet, with 10 Gbps over 40Km

System-Area Networks



中山大學
SUN YAT-SEN UNIVERSITY

- Interconnects within one “machine”
  - Interconnect in a multi-processor system
  - Interconnect in a supercomputer
  - Hundreds to thousands of devices interconnected
    - Tianhe-2 supercomputer (16K nodes)
- Maximum interconnect distance
  - Fraction to tens of meters (typical)
  - A few hundred meters (some)
    - InfiniBand: 120 Gbps over a distance of 300m

On-Chip Networks



中山大學
SUN YAT-SEN UNIVERSITY

- Interconnect within a single chip
  - Devices are micro-architectural elements
    - Caches, directories, processor cores
  - Currently, designs with tens of devices are common
    - Ex: IBM Cell, Intel multicores, Tile processors
- Proximity: millimeters

We are concerned with On-Chip and System-Area Networks

Why Study Interconnects?



中山大學
SUN YAT-SEN UNIVERSITY

- Provide external connectivity from system to outside world
  - Also, connectivity within a single computer system at many levels
- Interconnection networks should be well designed
  - To transfer the maximum amount of information
  - Within the least amount of time (and cost, power constraints)
- Application:
  - managing communication can be critical to performance

Why Study Interconnects? (cont.)



中山大學
SUN YAT-SEN UNIVERSITY

- Trends: high demand on communication bandwidth
  - increased computing power and storage capacity
  - switched networks (using network switches（交换机）) are replacing buses (e.g., Ethernet networks using hubs （集线器）)
- Computer architects/engineers must understand interconnect problems and solutions
  - in order to effectively design and evaluate systems

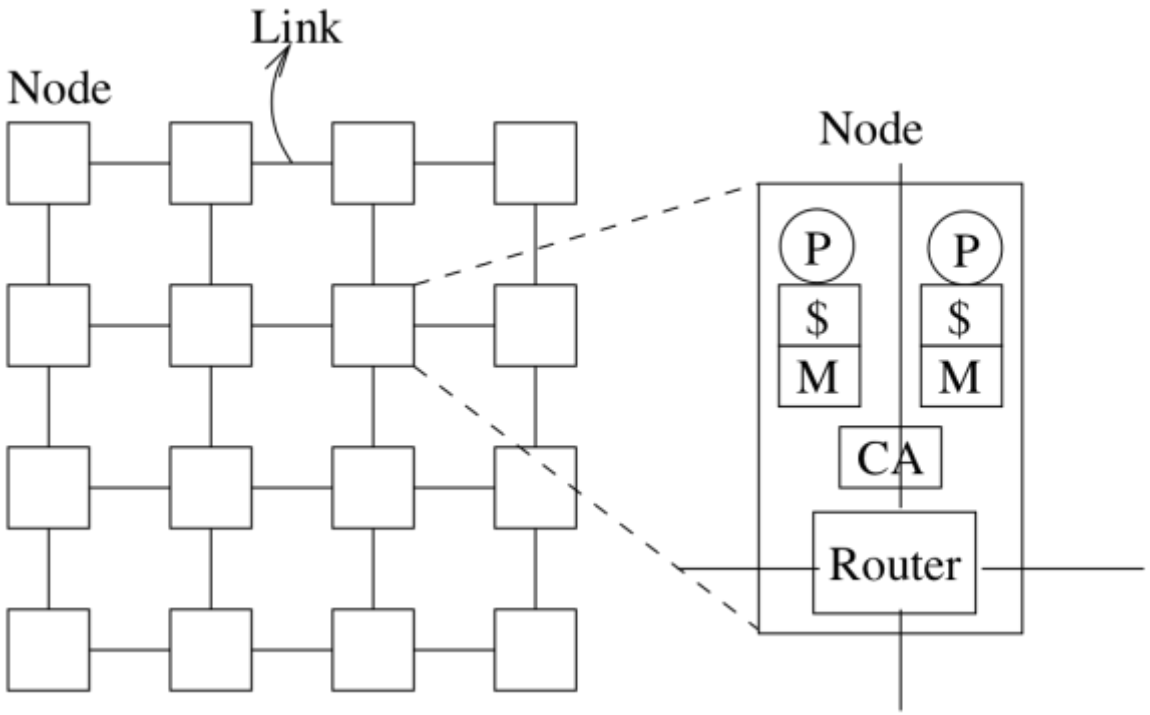
What we care about ICN

- Characteristics of communication
  - Latency must be low
  - Bandwidth must be high
  - Message characteristics
    - Many small messages with several fixed sizes
    - The largest message is one cache block size (64 or 128 bytes)

- Implications
  - Only two layers are sufficient: link level and node level
  - Communication protocol must be simple
    - no packet dropping, no elaborate flow control

1. Network Topology

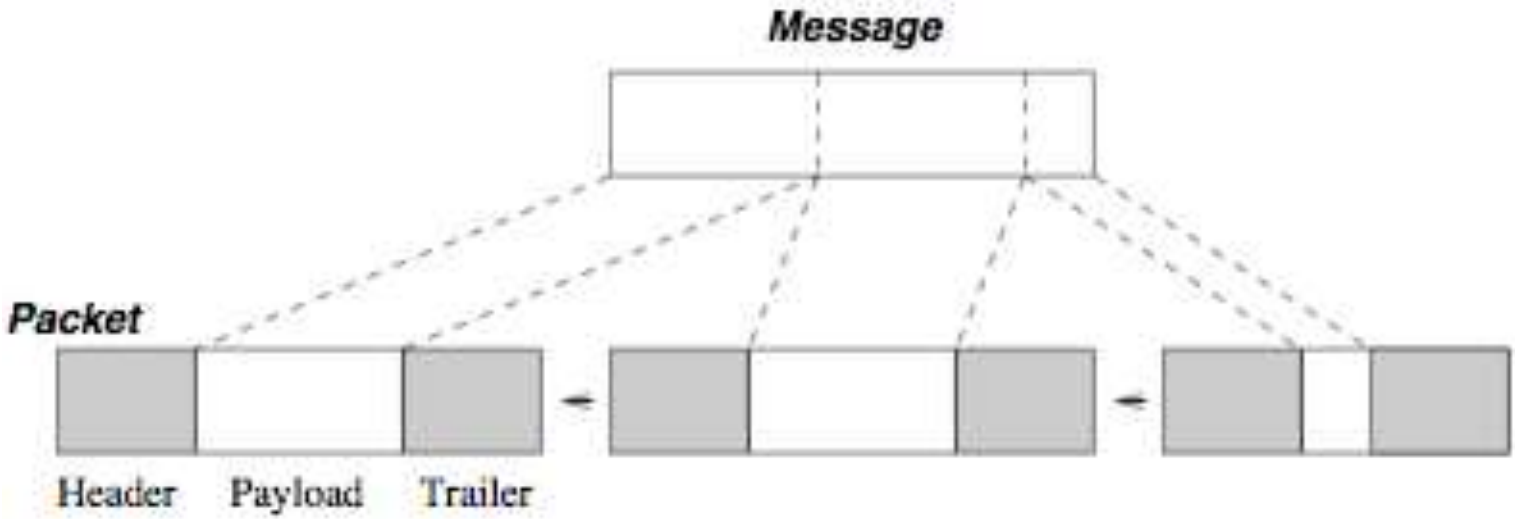
Nodes and Links



- **A node encapsulates**
  - one or more processors
  - communication controller/assist (CA)
  - router
  - Routers are connected by links
  - Links may be unidirectional or bidirectional

Link and Channel

- **Link = set of wires interconnecting a pair of nodes**
  - Link can be unidirectional or bidirectional
  - Unit of transfer determined by flow control protocol (flit)
  - Flit size determined by link latency and amount of buffering available
- **Channel = link + sender + receiver**
- A message transmitted over a channel is broken into **packets**
  - A packet has a header, trailer, and payload

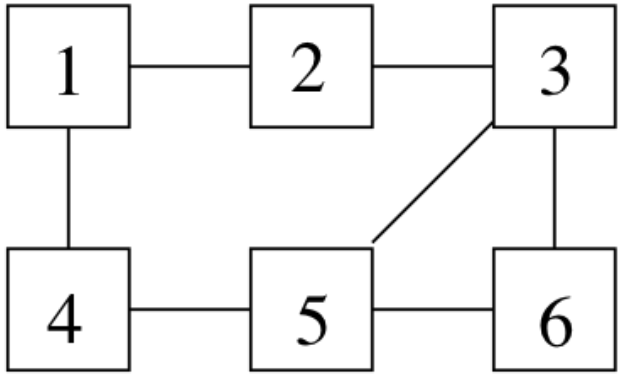


Network Topology

- **Network topology = shape of the network**
  - Determines the distance that a message needs to travel (in # links or network hops) from one node to another
- **Important characteristics**
  - **Diameter**: largest distance
  - **Average distance**
  - **Degree**: how many links are connected to a switch

Example

- **For the shown network, compute diameter, avg distance**



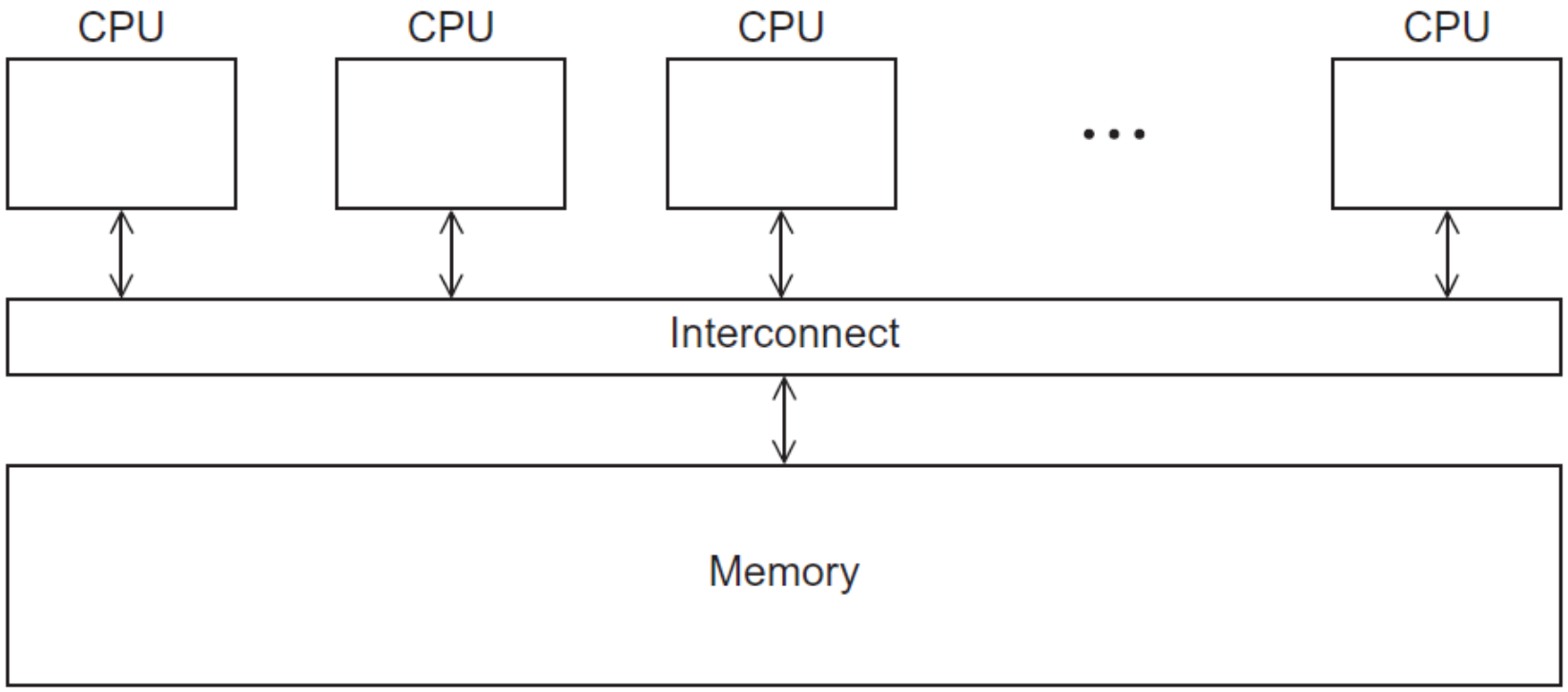
Diameter = 3

e.g. the distance between node 1 and 6

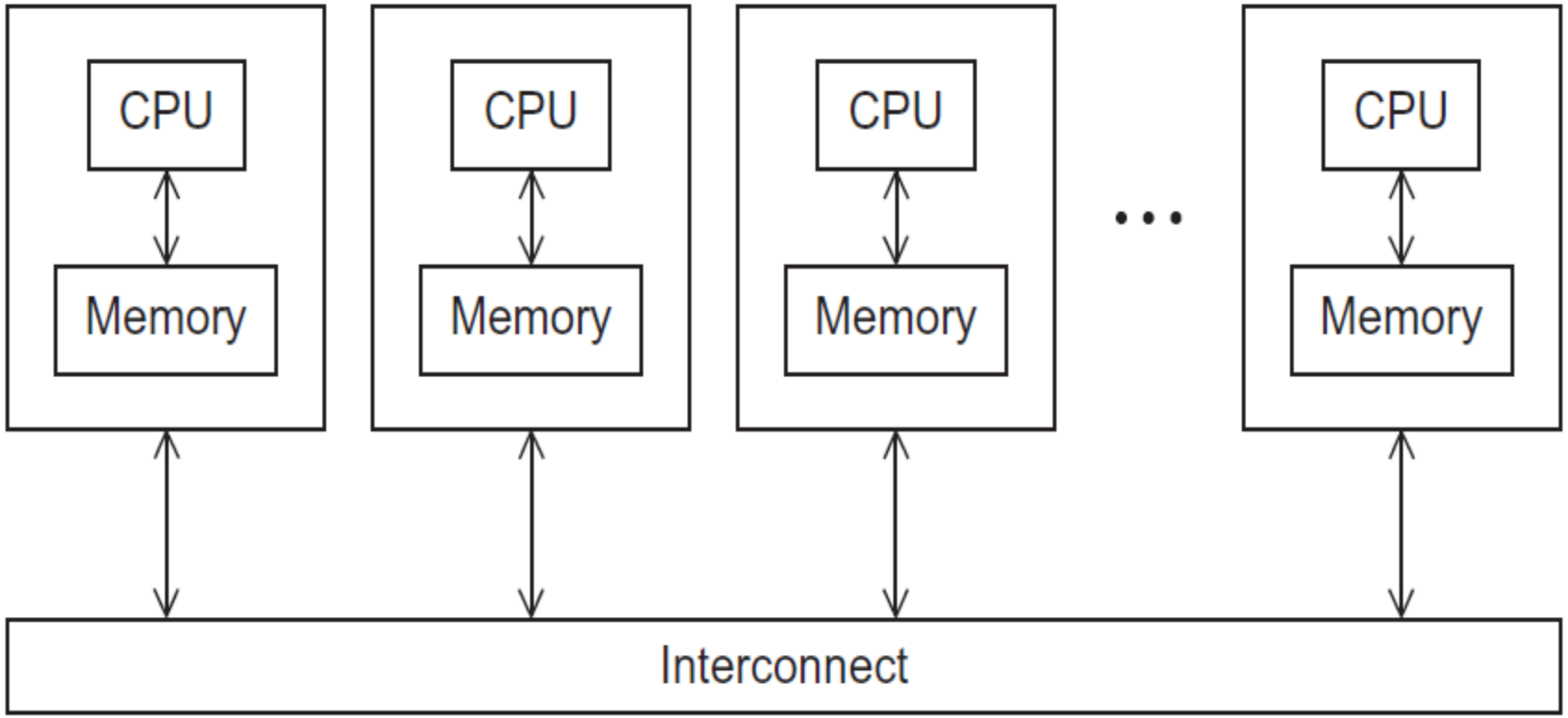
Avg distance:

d(1,2)=1, d(1,3)=2, d(1,4)=1, d(1,5)=2, d(1,6)=3
d(2,3)=1, d(2,4)=2, d(2,5)=2, d(2,6)=2
d(3,4)=2, d(3,5)=1, d(3,6)=1
d(4,5)=1, d(4,6)=2
d(5,6)=1
Sum of all distances = 24, number of node pairs = 15
Average distance = 24/15

Recall: Shared Memory System



Recall: Distributed Memory System



Interconnection networks

- Affects performance of both distributed and shared memory systems.

- Two categories:
  - Shared memory interconnects
  - Distributed memory interconnects

1.1 Shared memory interconnects

Shared memory interconnects

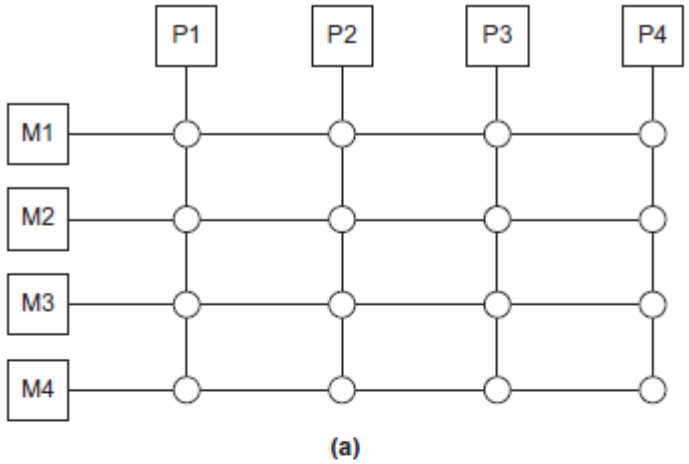
- **Bus interconnect**
  - A collection of parallel communication wires together with some hardware that controls access to the bus.
  - Communication wires are shared by the devices that are connected to it.
  - As the number of devices connected to the bus increases, contention for use of the bus increases, and performance decreases.

Shared memory interconnects

- **Switched interconnect**（交换互联网络）
  - Uses switches （交换器） to control the routing of data among the connected devices.
  - **E.g., Crossbar**（交叉开关矩阵）
    - Allows simultaneous communication among different devices.
    - Faster than buses.
    - But the cost of the switches and links is relatively high.

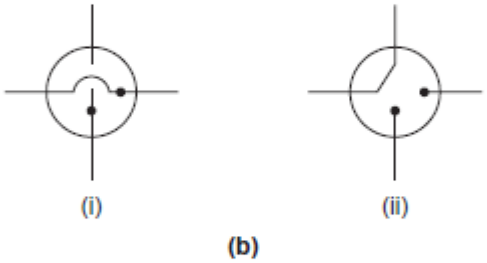
(a)

A crossbar switch connecting 4 processors (P_i) and 4 memory modules (M_j)

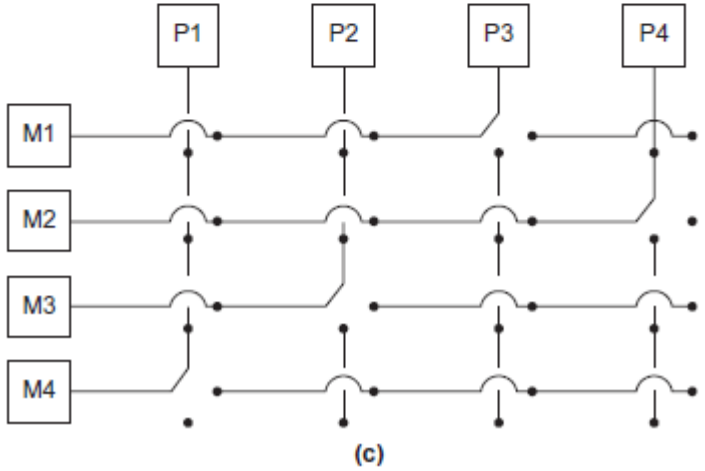


(b)

Configuration of internal switches in a crossbar



(c) Simultaneous memory accesses by the processors



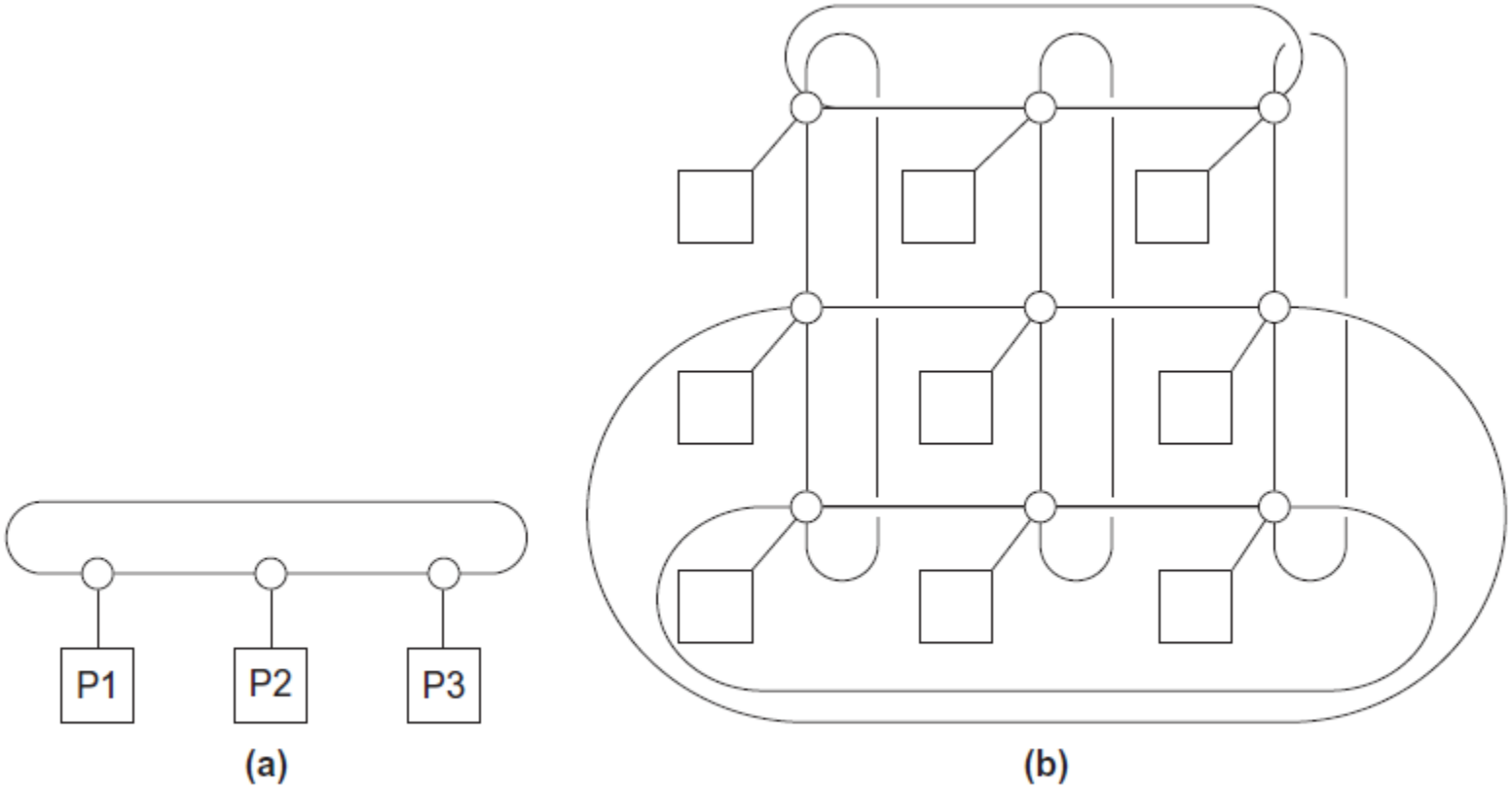
1.2 Distributed memory interconnects

Distributed memory interconnects

- **Two groups**
  - **Direct interconnect**
    - Each switch is directly connected to a processor memory pair, and the switches are connected to each other.
  - **Indirect interconnect**
    - Switches may not be directly connected to a processor.

(1) Direct interconnect

Direct interconnect



ring

toroidal mesh

（二维环面网格）

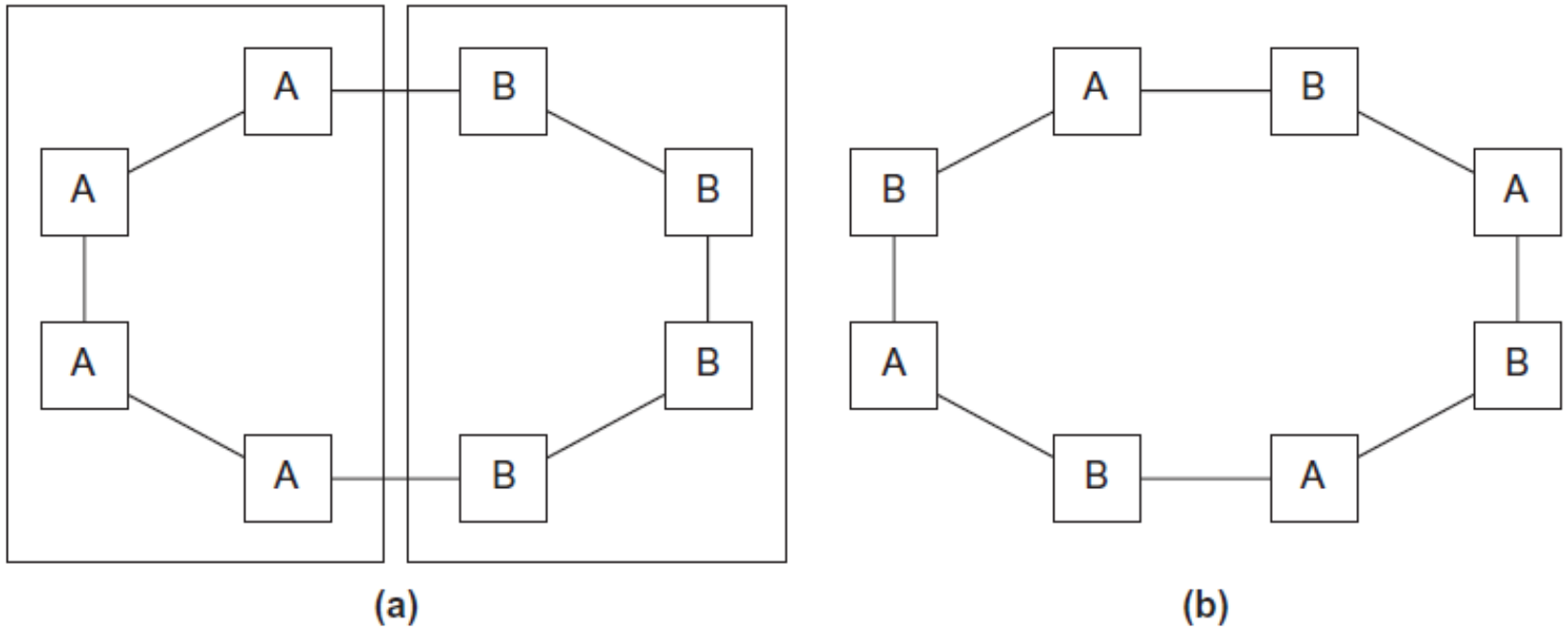
Bisection width（等分宽度）

- A measure of “number of simultaneous communications” or “connectivity”.

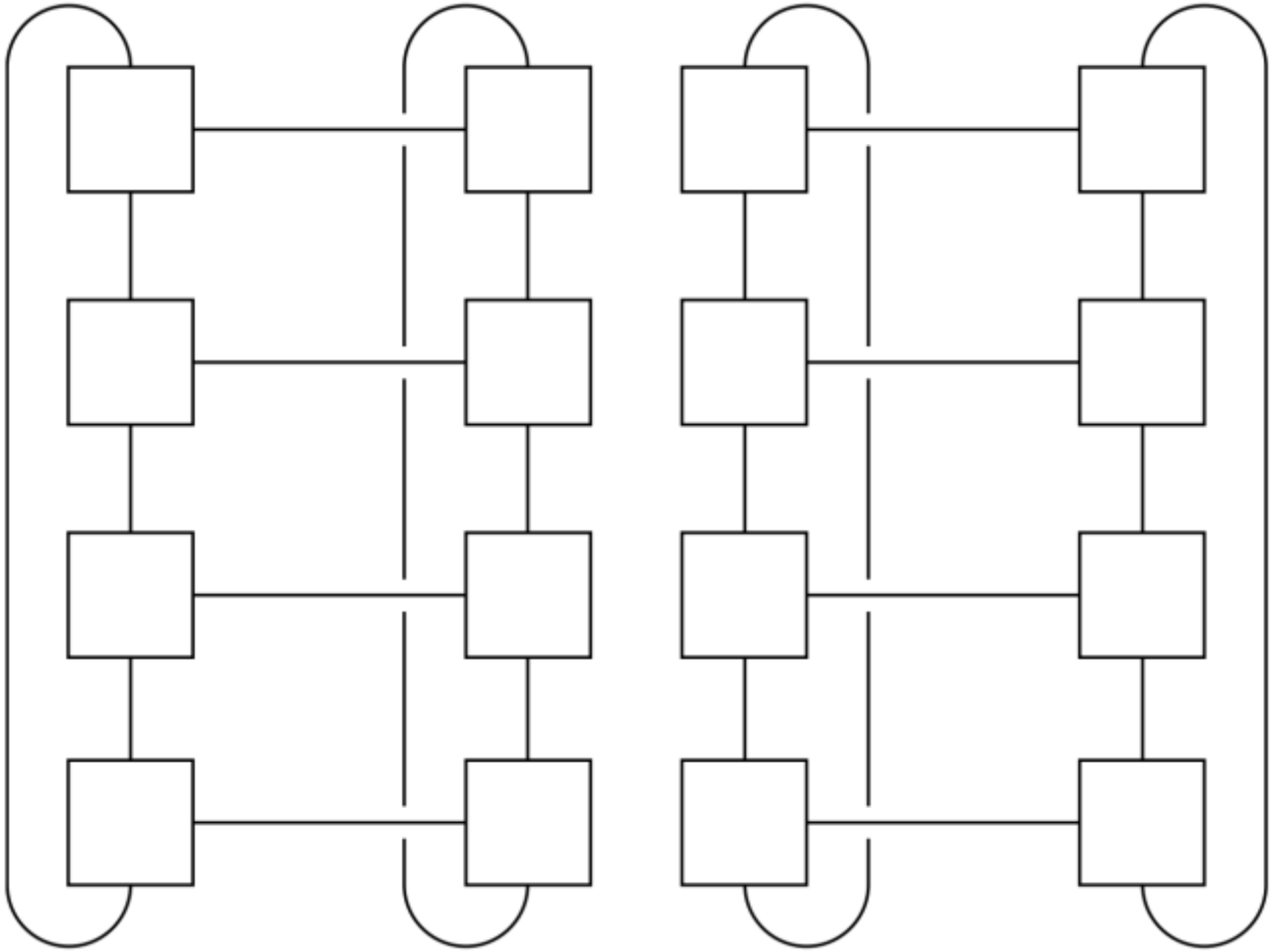
- How many simultaneous communications can take place “across the divide” between the halves?



Two bisections of a ring



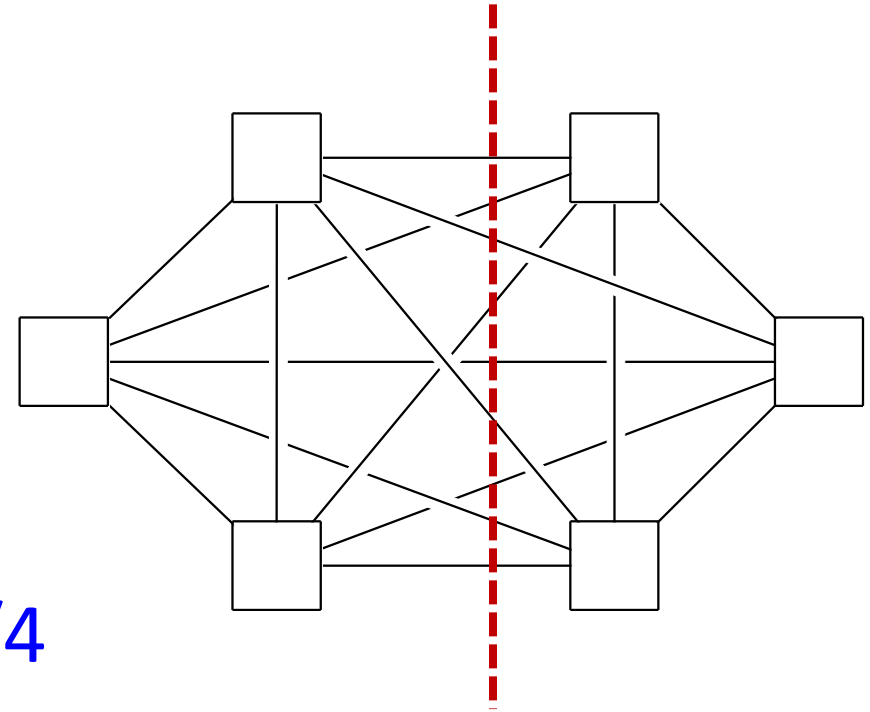
Computing bisection width



Fully connected network

- Each switch is directly connected to every other switch.

impractical



bisection width = $p^2/4$

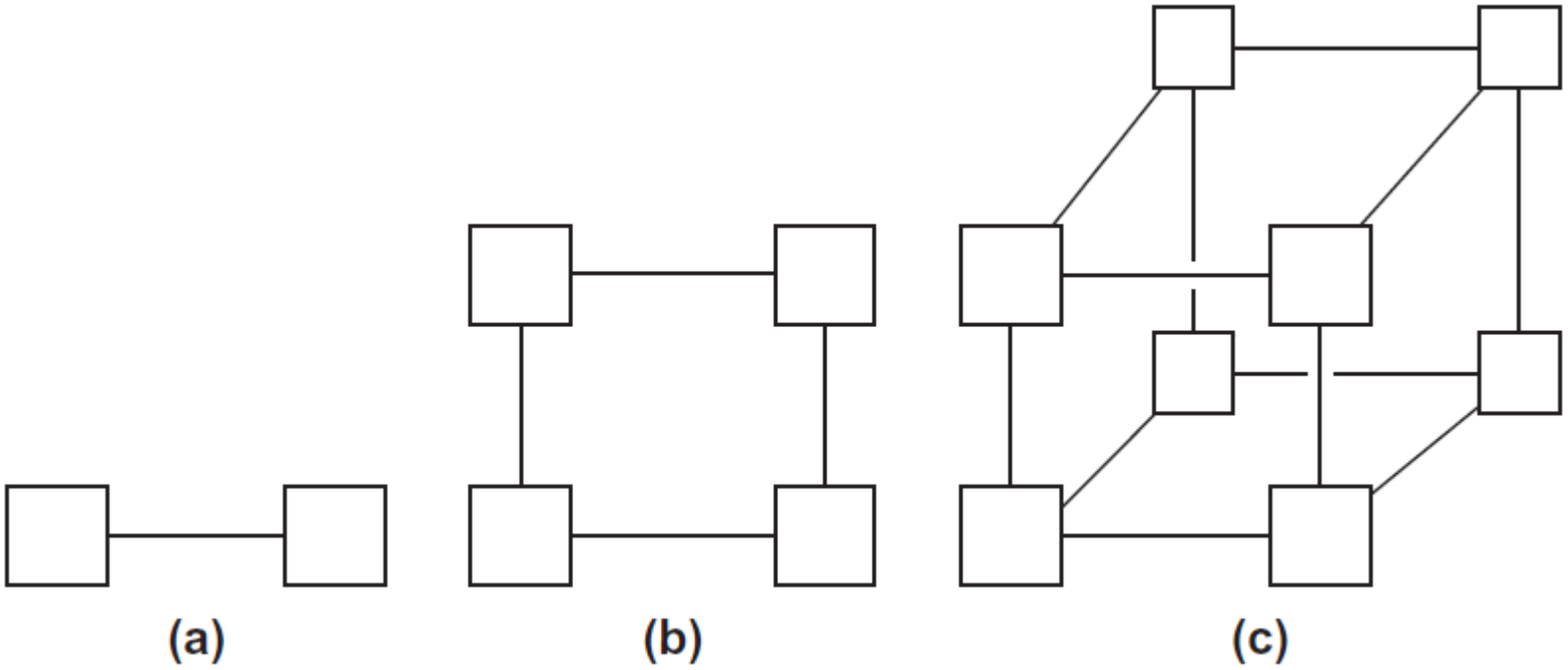
(p: # of nodes)

Hypercube（超立方体）

- Highly connected direct interconnect.

- **Built inductively:**
  - A one-dimensional hypercube is a fully-connected system with two processors.
  - A two-dimensional hypercube is built from two one-dimensional hypercubes by joining “corresponding” switches.
  - Similarly, a three-dimensional hypercube is built from two two-dimensional hypercubes.

Hypercubes



one- two- three-dimensional

Definitions

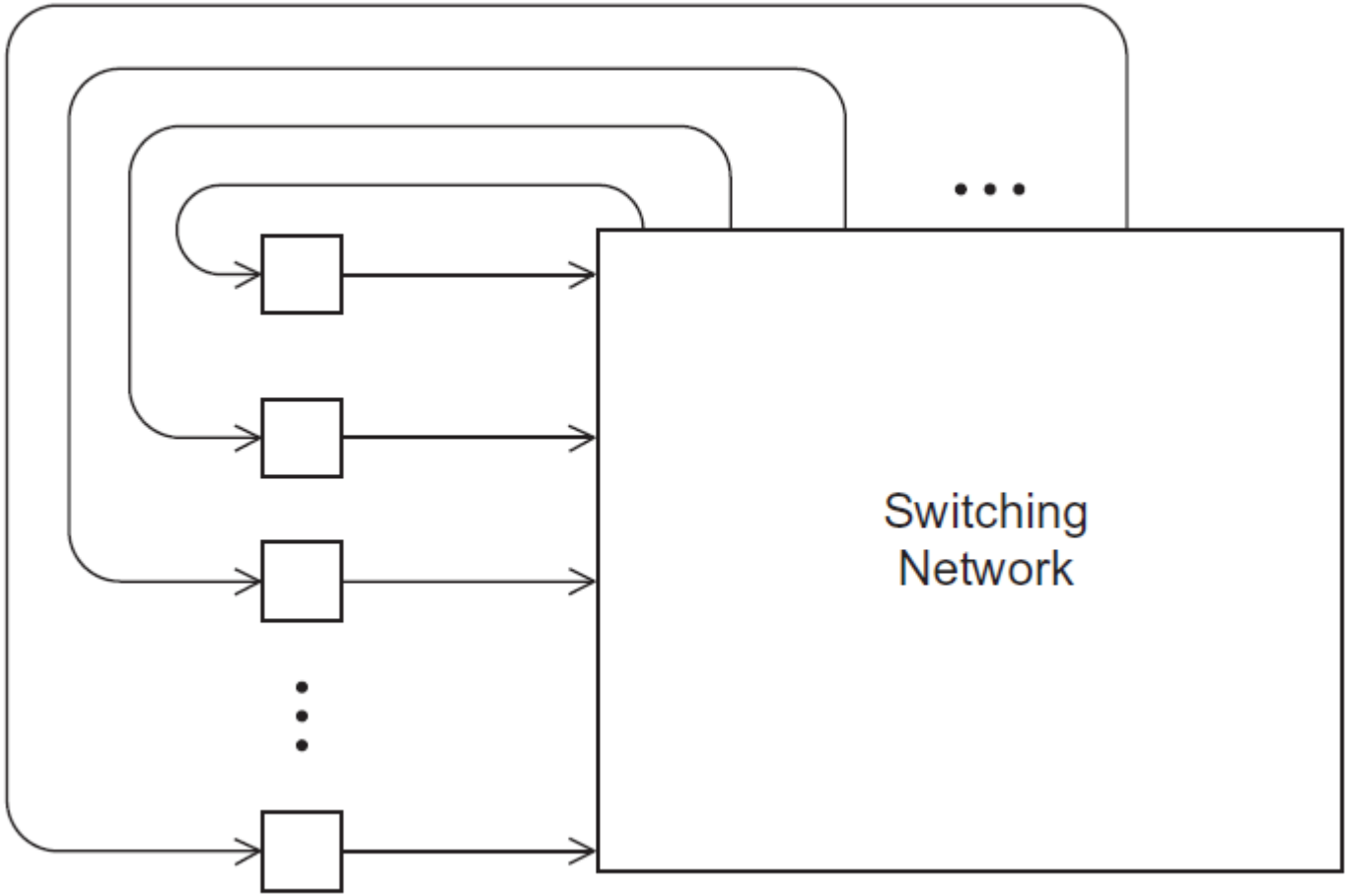
- Bandwidth
  - The rate at which a link can transmit data.
    - Usually given in megabits or megabytes per second.
- Bisection bandwidth
  - A measure of network quality.
  - Instead of counting the number of links joining the halves, it sums the bandwidth of the links.

(2) Indirect interconnect

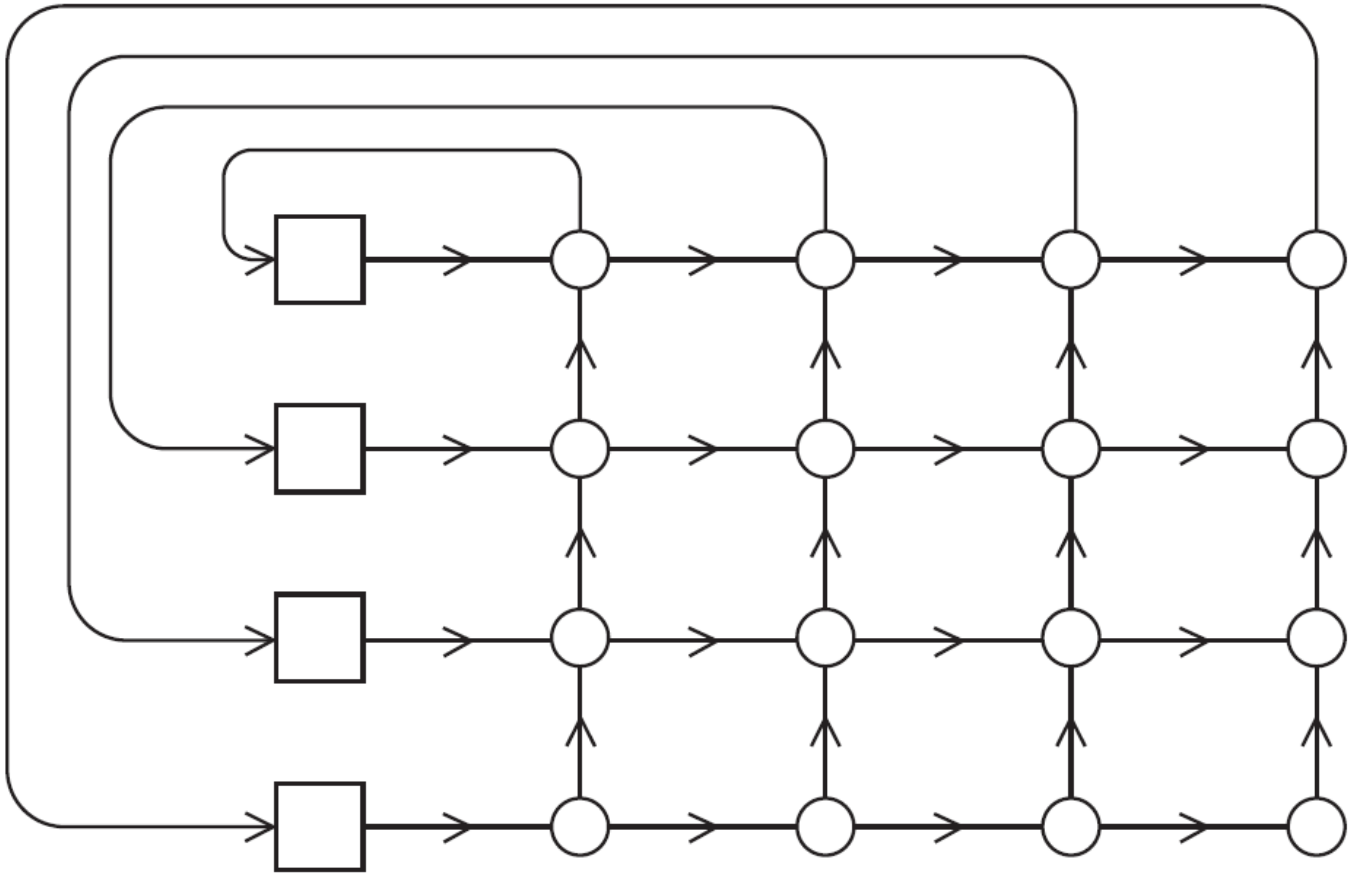
Indirect interconnects

- Often shown with unidirectional links and a collection of processors
- Each of which has an outgoing and an incoming link, and a switching network.
- Simple examples of indirect networks:
  - Crossbar （交叉开关矩阵）
  - Omega network

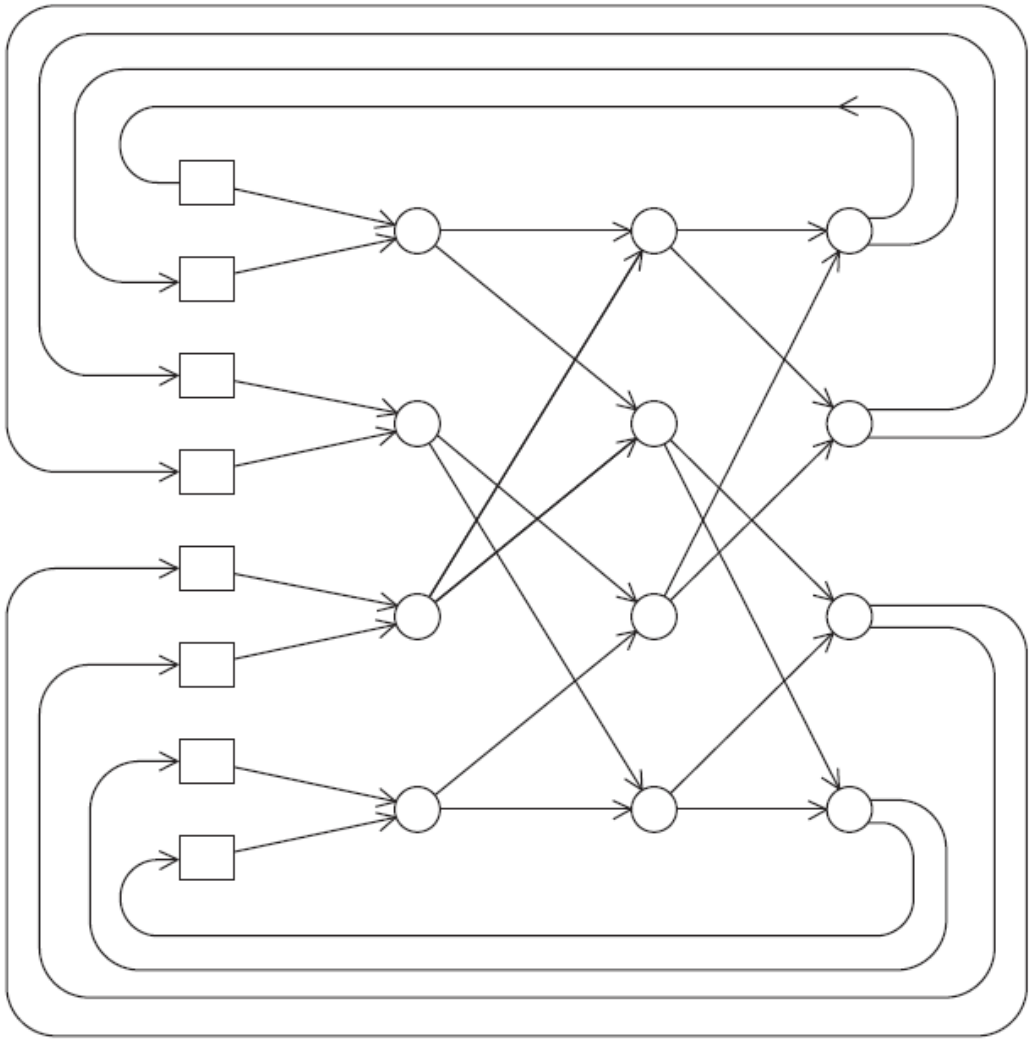
A generic indirect network



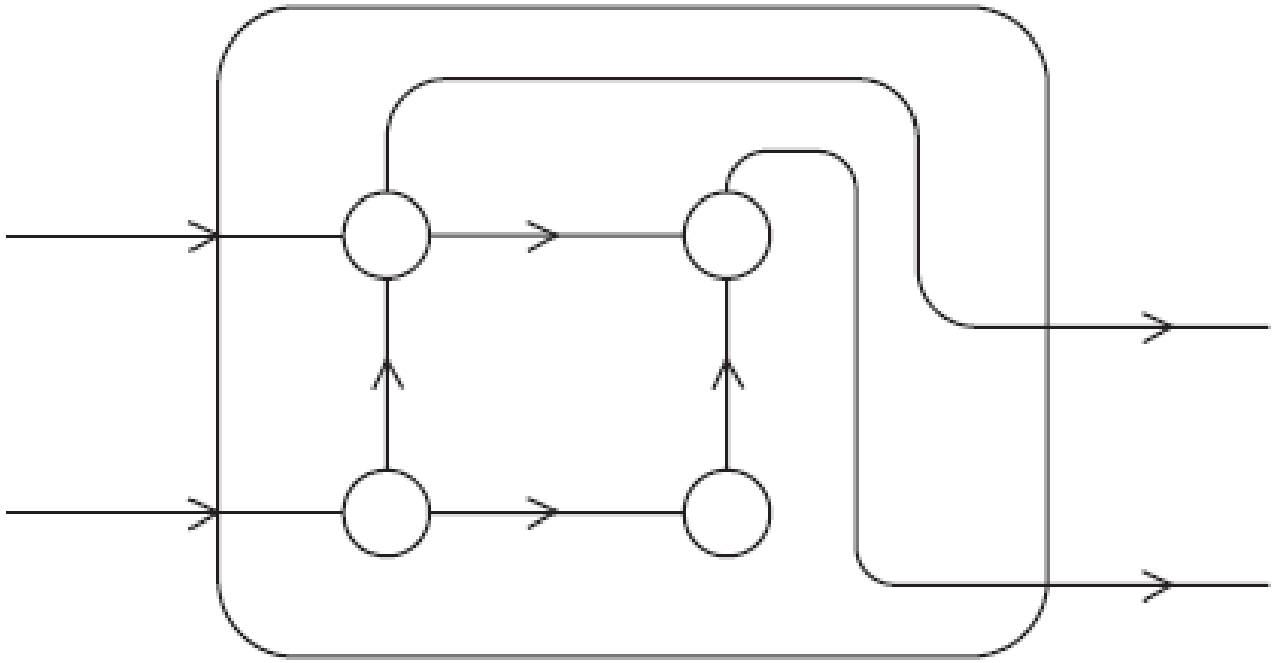
Crossbar interconnect for distributed memory



An omega network



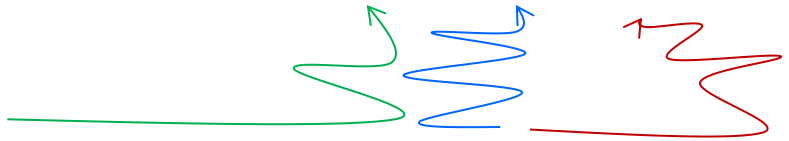
A switch in an omega network



More definitions

- Any time data is transmitted, we're interested in how long it will take for the data to reach its destination.
- Latency
  - The time that elapses between the source's beginning to transmit the data and the destination's starting to receive the first byte.
- Bandwidth
  - The rate at which the destination receives data after it has started to receive the first byte.

Message transmission time = `l + n / b`



latency (seconds)

length of message (bytes)

bandwidth (bytes per second)

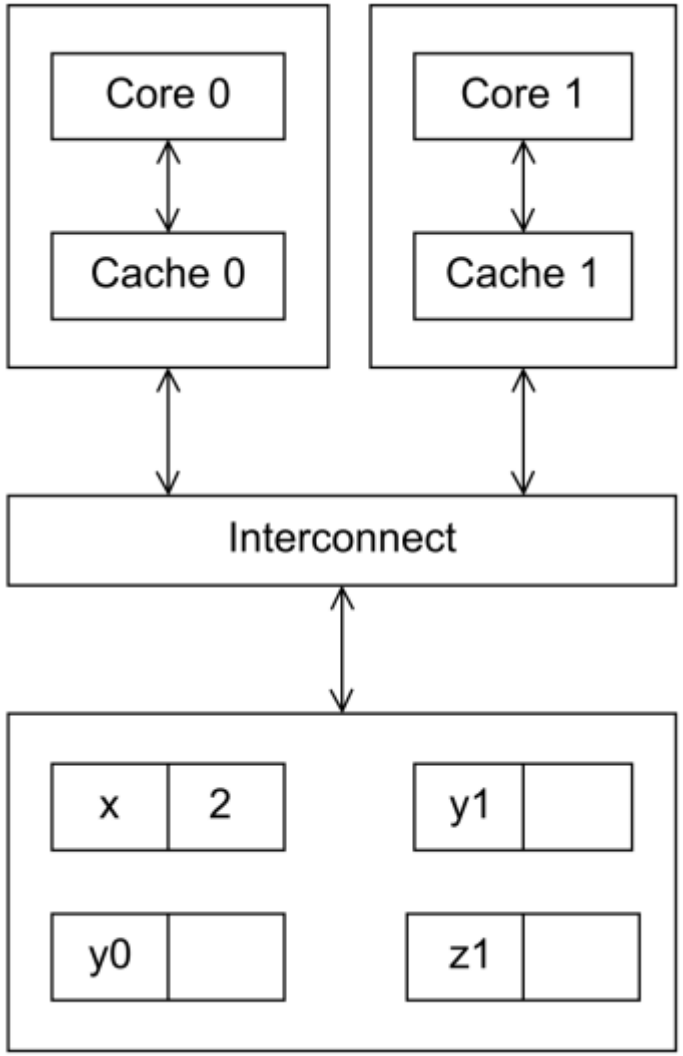
目录

- Classification of parallel systems
- Interconnection networks
- **Cache coherence**
  - 理解和运用基本机制

Recall that…

- **Caches are managed by system hardware**
- **Out of a programmer's direct control!**
- **Any undesired consequence?**

What would happen?



What would happen?

Time	Core 0	Core 1
0	`y0 = x;`	`y1 = 3*x;`
1	`x = 7;`	Statement(s) not involving `x`
2	Statement(s) not involving `x`	`z1 = 4*x;`

- **Suppose executed at the indicated times**
- **What value z1 will get, 4*7=28 or 4*2=8?**
- **Unpredictable! Whether write-through or write-back…**

Cache Coherence Problem

- When the caches of multiple processors store the same variable, an update by one processor to the cached variable, how to ensure the cached value stored by the other processors is also updated.

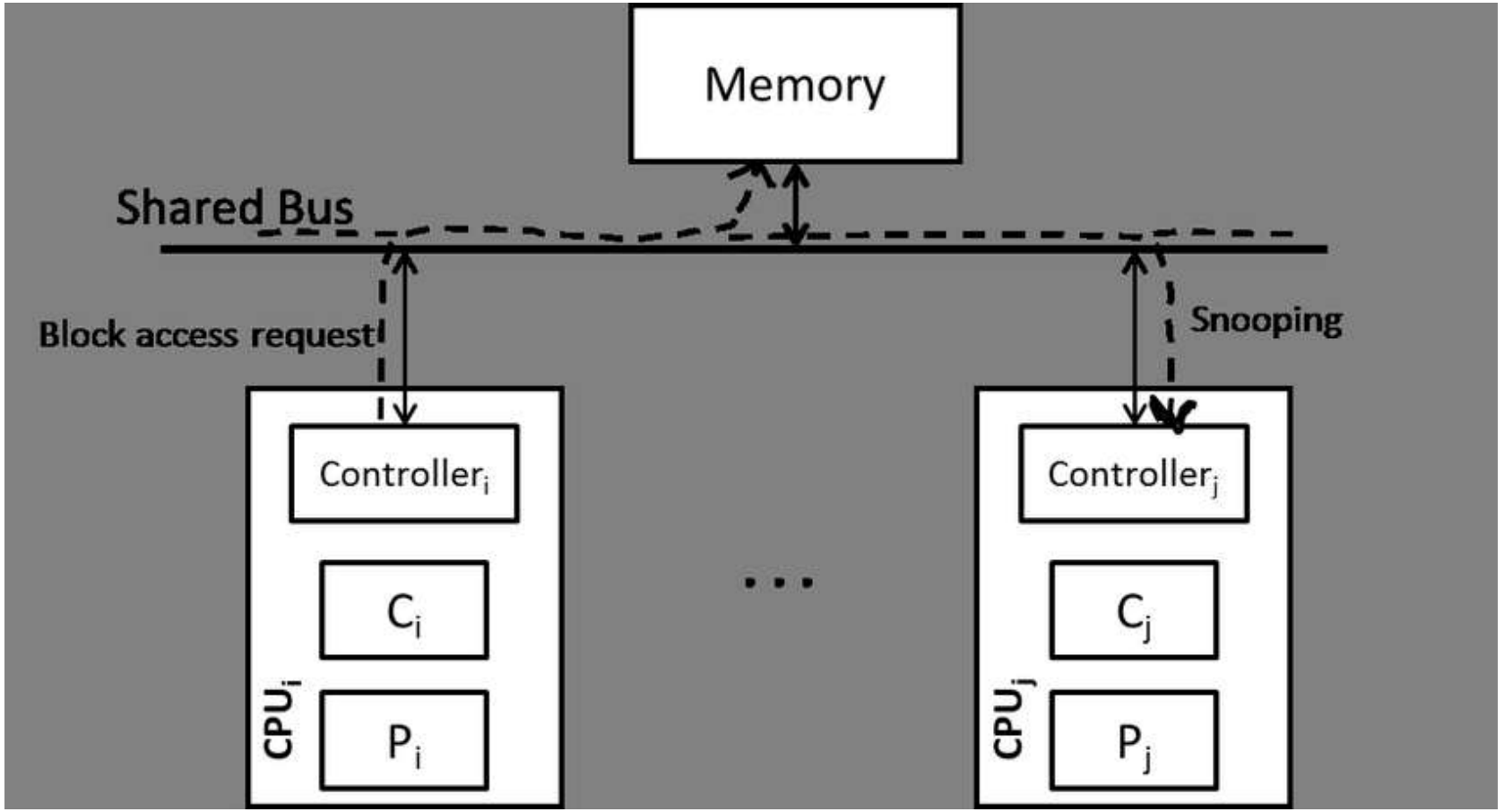
Cache Coherence Protocol

- **A cache coherence protocol specifies how reads, writes, invalidations, and updates are coordinated so that when one core modifies a data item, other cores do not keep using stale copies.**
  - Works cache line by cache line
  - Every private cache has a state for each cached block
  - Two kinds of events cause state transitions

Cache Coherence Protocol~

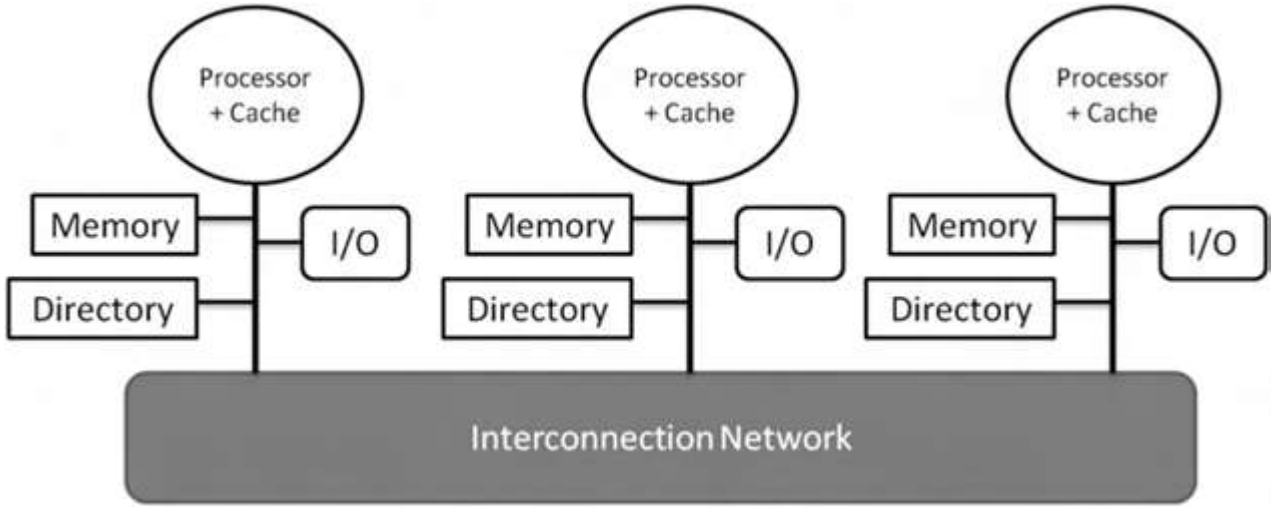
- Every private cache has a state for each cached block
  - whether the block is valid locally, whether other caches may also have it, whether memory is up to date, and who should respond if another core asks for the block.
  - I (Invalid), S (Shared), M (Modified), O (Owned), F (Forward)
- Two kinds of events cause state transitions
  - Local processor actions on this core
    - E.g., read hit, read miss, write hit, write miss, eviction.
  - Remote coherence actions caused by other cores
    - E.g., another core requesting a shared copy, requesting exclusive write permission, or sending invalidations.

监听（Snooping）Cache Coherence

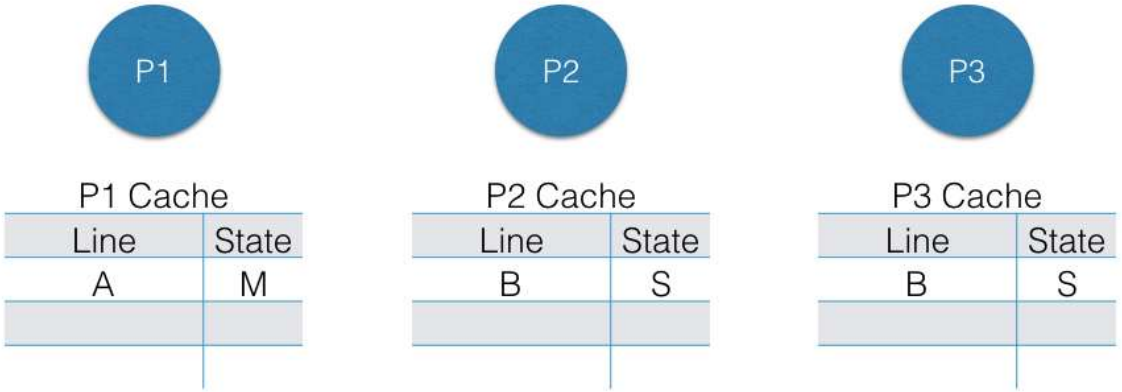


基于目录的(Directory-based) Cache Coherence

- **Directory (a data structure) stores the status of each cache line**
  - Also support distributed-memory
  - Directory is maintained in a distributed way



基于目录的(Directory-based) Cache Coherence



Location	Nodes	State
A	100	EM
B	011	S
C	000	U
D	000	U
…		

Diagram of full bit vector directory format, where E=Exclusive, S=Shared, M=Modified, and U=Uncached

基于目录的(Directory-based) Cache Coherence

- **Cons: Substantial additional storage required for the directory**

- **Pros: When a cache variable is updated, only the cores storing that variable need to be contacted**

伪共享 (False Sharing)

```
/* Private variables */
int i, j, iter_count;

/* Shared variables initialized by one core */
int m, n, core_count
double y[m];

iter_count = m/core_count

/* Core 0 does this */
for (i = 0; i < iter_count; i++)
   for (j = 0; j < n; j++)
      y[i] += f(i,j);

/* Core 1 does this */
for (i = iter_count; i < 2*iter_count; i++)
   for (j = 0; j < n; j++)
      y[i] += f(i,j);
```

Suppose $m = 8$, core_count = 2, **each cache line is 64 bytes!**

The system is behaving *as if* the elements of y were being shared by the cores !

谢谢！

- Q&A

中山大學
SUN YAT-SEN UNIVERSITY